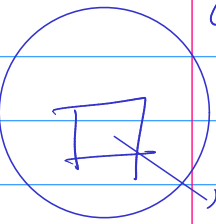


Chapter 1

IR Calculation

Wafer Area $\Rightarrow \pi r^2$



Die area $\Rightarrow$ Height $\times$ Width
Die

Die per wafer = $\frac{\text{Wafer Area}}{\text{Die area}}$

Cost per die $\Rightarrow \frac{\text{Cost per wafer}}{\text{Die per wafer} \times \text{yield}}$

Yield $\Rightarrow \frac{1}{\left(1 + (\text{Defect per area} \times \text{Die area})\right)^n}$

Working Dies $\Rightarrow$ Dies per wafer $\times$ yield

Question - 1:

Assume a 19 cm diameter wafer, X has a cost of 20, contains 89 dies, and has 0.023 defects/cm². Assume another 20 cm diameter wafer, Y has a cost of 15, contains 100 dies, and has 0.031 defects/cm². Consider N = 2.

- A. What do you understand by yield?
- B. Dies per wafer $\approx$ Wafer area / Die Area - Explain why there is " $\approx$ " not " $=$ "
- C. Find the yield for both wafers.
- D. Find the cost per die for both wafers.

$$e) \quad r \Rightarrow \frac{19}{2} \Rightarrow 9.5$$

$$\text{wafer area} \Rightarrow \pi \times (9.5)^2 \\ \Rightarrow 283.52$$

$$\text{Die area} \Rightarrow \frac{283.52}{89} \Rightarrow 3.19$$

$$\text{Yield} = \frac{1}{\left(1 + (\text{Defect} \times \text{Die area})\right)^N} \\ \Rightarrow \frac{1}{\left(1 + (0.023 \times 3.19)\right)^2} \\ \Rightarrow 0.86$$

$$\text{Yield Y} \Rightarrow \frac{1}{\left(1 + (\text{Defect} \times \text{Die area})\right)^N} \\ \Rightarrow \frac{1}{\left(1 + (0.031 \times 3.14)\right)^2} \\ \Rightarrow 0.83$$

$$\text{Die area} \\ \Rightarrow \frac{314.15}{100} \\ \Rightarrow 3.14$$

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{total die per wafer}}$$

$$\Rightarrow \frac{20}{80 \times 0.86}$$

$$\Rightarrow 0.26$$

$$\text{Cost per die} \Rightarrow \frac{15}{100 \times 0.83}$$

$$\Rightarrow 0.18$$

Question 2:

Suppose gaming consoles PlayStation 5 and Xbox Series X use different implementations of AMD's Zen 2 architecture. The instructions they support can be divided into four classes according to their CPI (class A, B, C, and D). PlayStation has a clock rate of 2.7 GHz and the instruction classes have CPIs of 7, 2, 3, and 6 respectively whereas Xbox has a clock rate of 3.0 GHz and the instruction classes have CPIs of 5, 4, 2, and 1 respectively. Now suppose, a program has an instruction count of 1.0×10^6 , and the instructions are divided into classes as follows:

- 30% class A,
- 50% class B,
- 10% class C, and

10% class D.

Now answer the following questions:

1. Calculate how many more clock cycles per instruction on average does the PlayStation take compared to the Xbox?
2. Calculate the difference between the execution time in these two consoles in milliseconds
3. If the program runs on a reference PC with an execution time of 120 ms, calculate the SPECratio for the PlayStation

Execution time $\Rightarrow$ $\frac{\text{Inst Count} \times \text{Clock cycle}}{\text{Clock cycle time}}$

Class A $\Rightarrow$	3×10^5	7	5
Class B $\Rightarrow$	5×10^5	2	4
Class C $\Rightarrow$	1×10^5	3	2
Class D $\Rightarrow$	1×10^5	6	1

$$\text{Total clock cycle} \Rightarrow (3 \times 10^5 \times 7) + (5 \times 10^5 \times 2) + (1 \times 10^5 \times 3) + (1 \times 10^5 \times 6)$$

$$\Rightarrow 9 \times 10^6 \quad \text{Avg CPI} \Rightarrow \frac{9 \times 10^6}{1 \times 10^6} = 9$$

$$\text{Total cycle} \Rightarrow 3.8 \times 10^6$$

$$\text{Diff} = (9 - 3.8) \times 10^6 \Rightarrow 0.2 \times 10^6$$

$$\text{Execution time} \Rightarrow \text{Ins count} \times \text{Avg CPI} \times \text{Clock cycle time}$$

$$\Rightarrow 1 \times 10^6 \times 4 \times 10^6 \times \frac{1}{2.7 \times 10^9}$$

$$\Rightarrow 1481.48 \text{ ms}$$

$$\text{Execution time} \Rightarrow 1 \times 10^6 \times 3.8 \times 10^6 \times \frac{1}{3 \times 10^9}$$

$$\approx 1266.66 \text{ ms}$$

$$\text{Difference} \Rightarrow 214.82 \text{ ms}$$

$$\text{Speed Ratio} \Rightarrow \frac{\text{Ref time}}{\text{Execution time}}$$

$$\Rightarrow \frac{120}{1481.48}$$

$$\Rightarrow 0.081$$

Question - 3:

Consider three different processors P1, P2, and P3 executing the exact same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.

Now answer the following questions:

- Which processor has the highest performance expressed in instructions per second?
- If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.
- We are trying to reduce the execution time by 30%, but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?

P1A Execution time = $I \times 1.5 \times \frac{1}{3 \times 10^9}$

$$\Rightarrow I \times 5 \times 10^{-10}$$

PCB, $\Rightarrow I \times 4 \times 10^{-10}$

PC3 $\Rightarrow I \times 5.5 \times 10^{-10}$

Let's take Inst count, $I = 1$ as same ISA

Per A = $\frac{1}{5 \times 10^{-5}} = 2 \times 10^9$

Per B $\Rightarrow \frac{1}{4 \times 10^{-10}} \Rightarrow 2.5 \times 10^9$

Per C $\Rightarrow \frac{1}{5.5 \times 10^{-10}} \Rightarrow 1.8 \times 10^9$

$\therefore$ B is more powerful

$$b) \text{Execution time} = \text{Inst count} \times \text{Avg CPI} \times \text{Clock cycle time}$$

$$10 \times 10^3 \Rightarrow \text{Inst count} \times 1.5 \times \frac{1}{3 \times 10^9}$$

$$\text{Inst count} \Rightarrow \frac{10 \times 3 \times 10^9}{1.5} \Rightarrow 2 \times 10^{10}$$

$$\text{Inst count} \Rightarrow 10 \times 2.5 \times 10^9 \Rightarrow 2.5 \times 10^{10}$$

$$\text{Inst count} \Rightarrow \frac{10 \times 4 \times 10^9}{2.2} = 1.8 \times 10^{10}$$

$$\text{Total cycle} \Rightarrow \text{Clock per Inst} \times \text{Inst Count}$$

$$\Rightarrow 3 \times 10^{10}$$

$$\Rightarrow 2.5 \times 10^{10}$$

$$\Rightarrow 3.94 \times 10^{10}$$

$$c) \text{Execution time} \Rightarrow 7 \quad \text{CPI} (1.8, 1.2, 2.64)$$

$$\text{Exec} = \text{Inst Count} \times$$

Question - 4:

Consider a computer running a program that requires 250s, with 70s spent executing add instructions, 85s executed sub instructions, and 40s spent executing left-shift instructions.

- By how much is the total time reduced if the time for add operations is reduced by 20%?
- Can the total time can be reduced by 20% by reducing only the time for left-shift instructions?

a)

$$56 + 85 + 40 = 181$$
$$\Rightarrow 69$$

$\Rightarrow$ 181

$p = C \times V + F$

~~$0.85 \times (0.814) + F$~~

~~$p + F + F$~~

$\Rightarrow 0.85 \times (0.81)$

$\Rightarrow 0.6885$

- b) Suppose a program is running on a PC with 3.2Ghz AMD Ryzen 5 processor. The program consists of 3 major types of instructions. Their instruction counts and CPI are given below:

Instruction Type	Instruction Count($\times 10^9$)	CPI
Load	3	2.3
Sub	5	?
Add	2	2.0

If the average CPI of the program is 6.3 then **what** is the CPI of the Sub Instruction?

3

- c) Based on the results of the SPEC CPU2000 benchmark conducted on an Intel Core i5 13th Gen processor, the system has an average CPI of 2.4 and an instruction count of 5×10^8 . The system completes 5,4000 clock cycles in 12 seconds, and the reference time is 9,650 seconds.

Find the SPEC ratio. Based on your findings, make a comment on the performance of the processor.

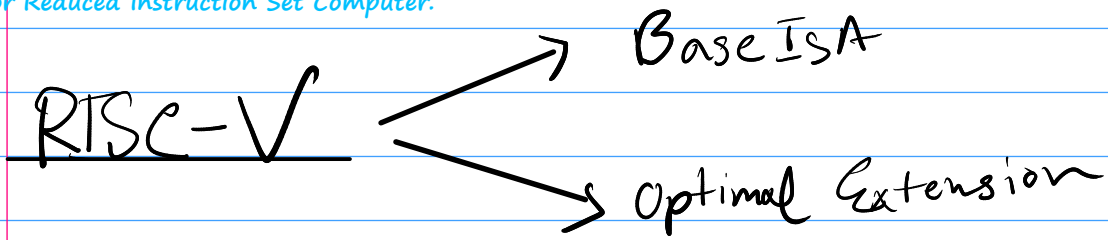
3

$$\text{Total Ins Count} \Rightarrow (3 + 5 + 2) \times 10^9 \\ \Rightarrow 10 \times 10^9$$

$$6.3 = \frac{(3 \times 10^9 \times 2.3) + (5 \times 10^9 \times x) + (2 \times 10^9 \times 2)}{10 \times 10^9}$$

$$\Rightarrow 10.42$$

RISC stands for Reduced Instruction Set Computer.



RISC V has 32×64 bit register

Size of individual register $\rightarrow$

$\downarrow$

Number of register

What do you understand by 64-bit architecture?

= Each register size is 64 bit,

2^{64} possible address,

Device can process 64bit data at a time

Unit

Double word (D) $\rightarrow$ 64 bit

word (w) $\rightarrow$ 32 bit

Half word (Hw) $\rightarrow$ 16 bit

Byte (B) $\rightarrow$ 8 bit

Registers in details ;

Register Number	Functionality
x0	Holds constant value 0. (Zero register)
x1	Stores the return address where you should return to after a function call is executed (Return Address register)
x2	Stores an address in the memory with the top of the stack. (Stack Pointer register)
x3	Global Pointer register.
x4	Thread Pointer register.
x5 to x7 and x28 to x31	Temporary register.
x8	Frame Pointer register.
x9, x18 to x27	Saved Register.
x10 to x11	Function arguments/ Return Values
x12 to x17	Function arguments

x₀ → always hold 0

x₁ → Return address

main() :

calc()

calc() :

return

→ x₁ save this line address and return in this after the function executed.

x₂ : Denotes top of the stack address.

$x_5 - x_7, x_{28} - x_{31}$: temp reg

$X_9, X_{18} - X_{27}$: Saved reg

$X_{10}, X_{11} \Rightarrow$ function Arg/return value

$x_{12} - x_{17} \rightarrow \text{function Ang}$

Operand

1. Register Operand
2. Memory Operand
3. Immediate Operand

Register operand: data is stored into the register and ready to perform the operation

add x_{20}, x_5, x_7
 $\downarrow \quad \quad \downarrow \quad \downarrow$
 $\alpha = 5 + 3$

Memory operand: load data from memory to register to perform operations

ld x21, 20(x7)

step 1: fetch data from 20(x7)

into x21

↓

20x^{64/8}

→ 20x8

Immediate operand: Constant data specified in the instruction

qddi $x_{20}, x_7, 10$

In RISC-V every address cell capacity is 8 bit

so for Double word (64 bit) data storing $64/8 \Rightarrow 8$ slot

for word (32 bit) data storing $32/8 \Rightarrow 4$ slot

for half word (16 bit) data storing $16/8 \Rightarrow 2$ slot

for byte (8 bit) data storing $8/8 \Rightarrow 1$ slot

No Alignment Restriction

Alignment

0
|
for 32 bit
need $(8 \times 8) = 64$ bit
capacity; $(64 - 32)$
7
 $\Rightarrow 32$ bit waste

Non-Alignment

0
|
for 32 bit need only
 $(8 \times 4) = 32$ bit space
3
hence; no space wasted

Endianness

① Little endian $\rightarrow$ LSB stores in lower add

② Big endian $\rightarrow$ MSB stores in lower add

moreover, RISC-V follows little endian.

advantage: In terms of addition, LSB comes first in 64 bit addition it fetch only 32 bit and perform addition. If the 32 bit part doesn't have any carry bit, then it only does 32 bit operation and this reduce redundancy.

Load Instruction:

LD
1
2
3
4
5
6
7
8

destination

offset

Source register

Load data from instruction

Base Relative addressing

Base + offset
↓
location of 0th index

Load means fetch data
from memory to reg

LD → Signed

LDU → Unsigned

~~Ex~~ $A[5] + g = a$

lw $X_7, 20(X_{22})$

Add X_{20}, X_7, X_{21}

$$a = X_{20}$$

$$g = X_{21}$$

$$A[0] = X_{22}$$

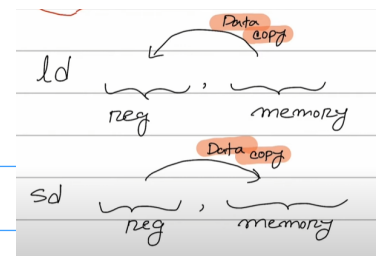
A is a word array

$$\therefore 32/8 \Rightarrow 4 \text{ words}$$

$$A[5] = 5 \times 4 \Rightarrow 20$$

Store Instruction:

Stores data from the register to memory.



SD _____, _____, (_____)
↓ Source → offset, Source
↓
W
↓
h
↓
B

Store

From

→

To

Reg

Memory

load

Memory

Register

~~Ex~~

$$A[8] = C + B[3]$$

$$32/8 \Rightarrow 4 \Rightarrow 4 \times 8 \Rightarrow 32$$

$$8/8 = 1 \Rightarrow 1 \times 3 = 3$$

lb x_5 , 3(x_{23})

Add x_6 , x_5 , x_{21}

Sw x_6 , 32(x_{22})

A is a word array.

B is a byte array.

$$C = x_{21}$$

$$A[0] = x_{22}$$

$$B[0] = x_{23}$$

Immediate Operand

Syntax, $\text{Addi} \underline{\hspace{2cm}}, \underline{\hspace{2cm}}, \underline{\hspace{2cm}}$
dest, source, constant

$a = b + 10$	$a = x_{21}$
$\text{addi } x_{21}, x_{22}, 10$	$b \Rightarrow x_{22}$

$a = b - 10$	$a = x_{21}$
$\text{addi } x_{21}, x_{22}, -10$	$b = x_{22}$

[because there are no subi in RISC-V]

Sign Extension

Unsigned Range $\Rightarrow 0 - (2^n - 1)$

for 8 bit $\Rightarrow 0 - (2^8 - 1)$
 $\Rightarrow 0 - 255$

Signed Range $\Rightarrow -2^{n-1} \text{ to } +2^{n-1} - 1$

$\Rightarrow -128 \text{ to } +127$

A	B	
0	0	0
0	1	0
1	0	0
1	1	1

And Instruction

we use AND for masking

we use OR for bit include

we use XOR for not and buffer

something and 0 = 0

something and 1 = something

x16 = 0000 0000 0110 0011 0101

x17 = 0000 0000 0001 1100 0110

x19 = 0000 0000 0000 0000 0100

Q. 1,2,6,7,8 no bit will be remain same others will be change to 0, store it to x19

and x19, x16, x17

des, s1, s2

AND, OR, XOR these are bit wise operation

means, the suppose 2 sources data are in x20, x21 and there and/or/xor result will be in x19 then there will be a bit operation between x20 and x21 and ans will be store in x19

OR Instruction

A	B	
0	0	0
0	1	1
1	0	1
1	1	1

something OR 0 = something

something OR 1 = 1

x16 = 0000 0000 0110 0011 0101

x17 = 0000 0000 0001 1100 0110

x19 = 0000 0000 0001 1100 0110

Q. 1,2,6,7,8 no bit will be

1 others will be remain same,
store it to x19

or x19, x16, x17

des, s1, s2

XOR Instruction

Something XOR 0 = something

something XOR 1 = Inverse of something

XOR x_9, x_{10}, x_{11}
des r1s1 r1s2

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

SLLI - Shift Left Logical Immediate

Valid range 1-63

because a reg capacity is 64 so if you shift more than 63 time there will be no remains; on the other hand if you do 0 times shift then the value will be as it is

Machine-code

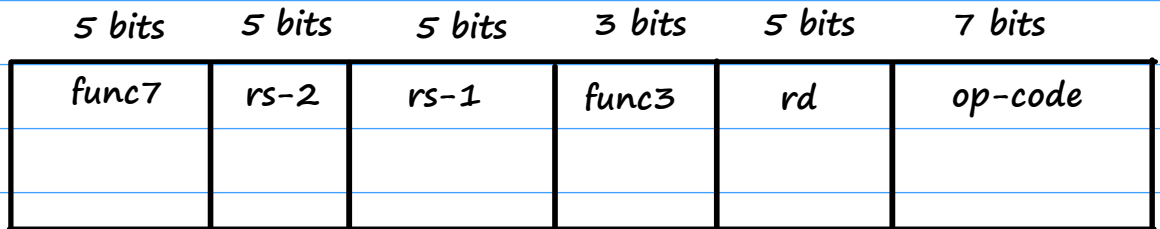
Total 32 bits

R-type => 1 destination, 2 source

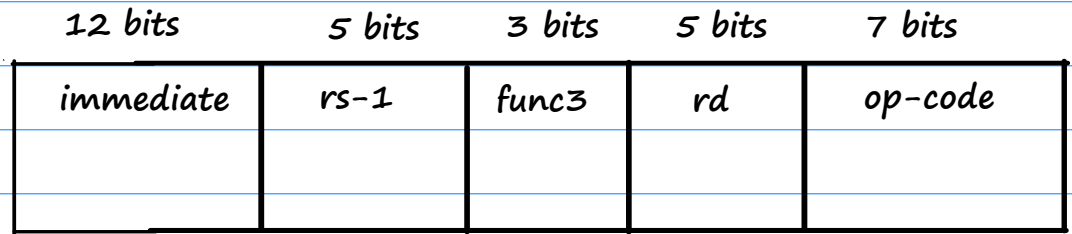
I-type => 1 destination, 1 source, Immediate value

S-type => 2 sources

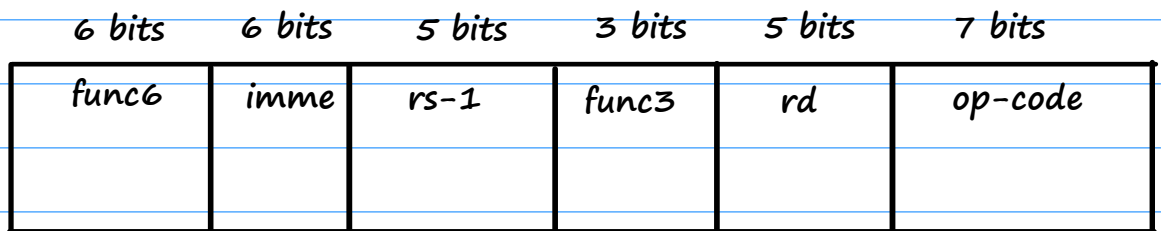
R-type



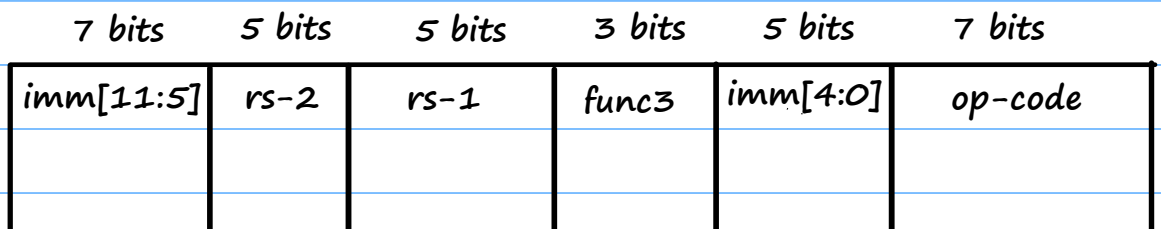
I-type



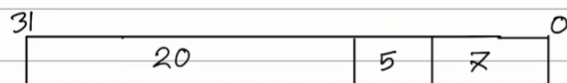
I-type(slli)



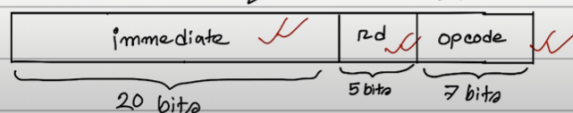
S-type



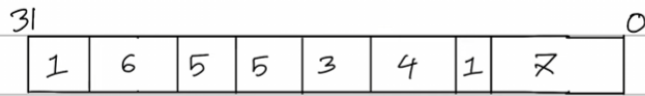
U type instruction → LUI



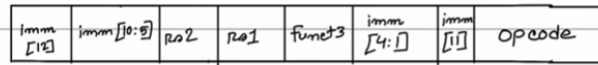
* each field has unique name and size.



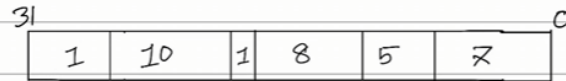
SB type instruction $\rightarrow$ Beq, Dne, Blt, Bge



* each field has unique name and size.



UJ type instruction - Jal



* each field has unique name and size.



HW 2

Q1 Data = 0000 0000 0000 0000 0000 0000 0000 0000
1100 0000 0011 0000 0011 0000 0001 0100

- a) How many bits can you store in 1 memory location/cell?
- b) Show how the given Data will be stored in memory of a RISC-V machine. Assume the starting memory location is 0
- c) Show how the given Data will be stored in memory of a machine that follows big endian. Assume the starting memory location is 8 approach

a) 8 bits

b) as RISC-V follows little endian method so LSB will store in lower number memory location

#0000 - 0001 0100

#0001 - 0011 0000

#0002 - 0011 0000

#0003 - 1100 0000

#0004 - 0000 0000

#0005 - 0000 0000

#0006 - 0000 0000

#0007 - 0000 0000

c) For Big endian approach

#0008 - 0000 0000

#0009 - 0000 0000

#0010 - 0000 0000

#0011 - 0000 0000

#0012 - 1100 0000

#0013 - 0011 0000

#0014 - 0011 0000

#0015 - 0001 0100

Q2 Data = (AB13579CD)₁₆

- a) How many bits can you store in 1 memory location/cell?
- b) Show how the given Data will be stored in memory of a risc-V machine. Assume the starting memory location is 0.
- c) Show how the given Data will be stored in memory of a machine that follows big endian. Assume the starting memory location is 12.

a) 8 bits

b)

A	B	1	3	5	7	9	C	D
1010	1011	0001	0011	0101	0111	1001	1100	1101

#0000 - 1100 1101

#0001 - 0111 1001

#0002 - 0011 0101

#0003 - 1011 0001

#0004 - 0000 1010

c) #0008 - 1010 1011

#0009 - 0001 0011

#0010 - 0101 0111

#0011 - 1001 1100

#0012 - 1101 0000

Q-3 Array A is a Byte array with size of 50. Base address of A is stored in X23. Variable g & h are associated with X5, X6 registers.
Write the necessary RISC-V code of the following code.

$$A[12] = A[5] + A[7] + g + A[2]$$

```

lb x6, 5(x23)    //A[5]
lb x7, 7(x23)    //A[7]
lb x19, 2(x23)   //A[2]
Add x18, x6, x7  //A[5] + A[7]
Add x18, x18, x19 //A[5] + A[7] + A[2]
Add x18, x18, x5 //x18 = A[5] + A[7] + A[2] + g
sb x18, 12(x23)  //A[12] = A[5] + A[7] + A[2] + g

```

05D0FE2300000000

Q-4 Data Memory: (little endian)

Location	Data	Loc	Data	Loc	Data	Loc	Data	X ₂₁ = (12) ₁₀
21	(AC) ₁₆	24	(D0) ₁₆	27	(00) ₁₆	30	(00) ₁₆	X ₂₂ = (13) ₁₀
22	(12) ₁₆	25	(FE) ₁₆	28	(00) ₁₆	31	(FF) ₁₆	
23	(05) ₁₆	26	(23) ₁₆	29	(00) ₁₆	32	(23) ₁₆	

Instructions:

LD X5, 10(X23)

After executing the above instruction, what is the value of X5?

Effective add = 10 + 12 = 23

//X22 = 13

#23 - 05

#24 - D0

X5 = 0000000023FED005

#25 - FE

#26 - 23

#27 - 00

#28 - 00

#29 - 00

#30 - 00

Reg $\Rightarrow$ LSB - MSB
same thing

Q.5 Data Memory: (little endian)

Location	Data	Loc.	Data	Loc.	Data	Loc.	Data	$X_{21} = (12)_{10}$
21	0011 1011	24	0000 1111	27	1111 1111	30	0011 0000	$X_{22} = (13)_{10}$
22	0011 0000	25	1011 0000	28	1011 0011	31	1000 0001	
23	0001 1111	26	0000 1011	29	0001 1000	32	1000 1111	

Instruction: $SB\ X_5, 10(X_{22})$ | $X_5 = 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011\ 1010$

msb
lsb

After executing the above instruction, update the data memory.

Effective address = $13 + 10 = 23$

#23 - 1011 1010

#24 - 0000 0000

#25 - 0000 0000

#26 - 0000 0000

#27 - 0000 0000

#28 - 0000 0000

#29 - 0000 0000

#30 - 0000 0000

HW 3

1. Add X12, X13, X15;

a) Convert the given instruction to its corresponding machine code.

b) Now from the 32 bit binary number you got in question 1.a try to find the instruction itself again.

a)

7bit	5bit	5bit	3bit	5bit	7bit
func7	RS2	RS1	Func3	RD	OPCODE

SD X12, 16(X11);

a) Convert the given instruction to its corresponding machine code.

b) Now from the 32 bit binary number you got in question 2.a try to find the instruction itself again.

7bit	5bit	5bit	3bit	5bit	7bit
Imm[11:5]	RS2	RS1	Func3	Imm[4:0]	OPCODE

3. Sll X12, X13, X14;

a) Convert the given instruction to its corresponding machine code.

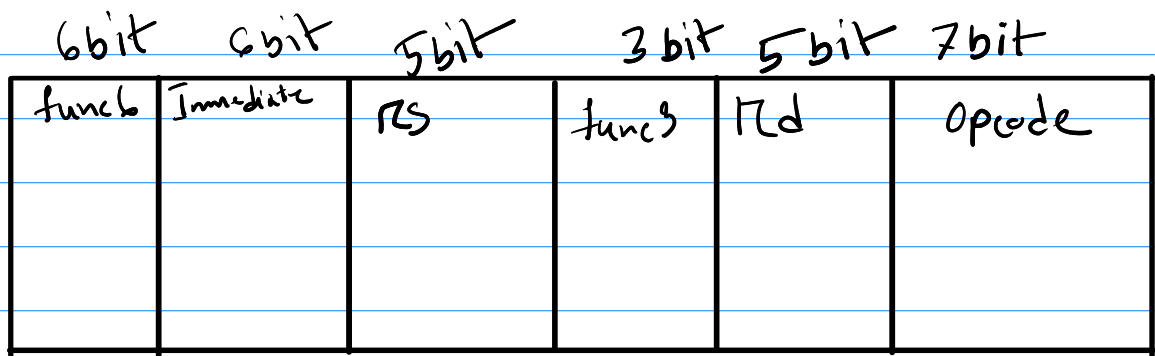
b) Now from the 32 bit binary number you got in question 3.a try to find the instruction itself again.

7bit	5bit	5bit	3bit	5bit	7bit
func7	RS2	RS1	Func3	RD	OPCODE

4. Slli X12, X16, 62;

a) Convert the given instruction to its corresponding machine code.

b) Now from the 32 bit binary number you got in question 4.a try to find the instruction itself again.



5. $a = 5b + 3c + (d/4)$;

$a \rightarrow X21$

$b \rightarrow X22$

$c \rightarrow X23$

$d \rightarrow X24$

Convert the given code to risc-v assembly instruction(s)

`srli x5, x24, 2` $//x5 = d/4$

`slli x6, x23, 1` $//x6 = 2c$

`add x6, x6, x23` $//x6 = 2c+c = 3c$

`slli x7, x22, 2` $//x7 = 4b$

`add x7, x7, x22` $//x7 = 4b+ b = 5b$

`add x21, x7, x6` $//x21 = 5b + 3c$

`add x21, x21, x5` $//x21 = 5b+ 3c + (d/4)$

6. $a = A[b] + 5;$

$a \rightarrow X_{21}$

$b \rightarrow X_{22}$

Base address of the double word array A is in X23

```
slli x5, x22, 3
```

```
add x6, x23, x5
```

$Id \times 5, O(x6)$

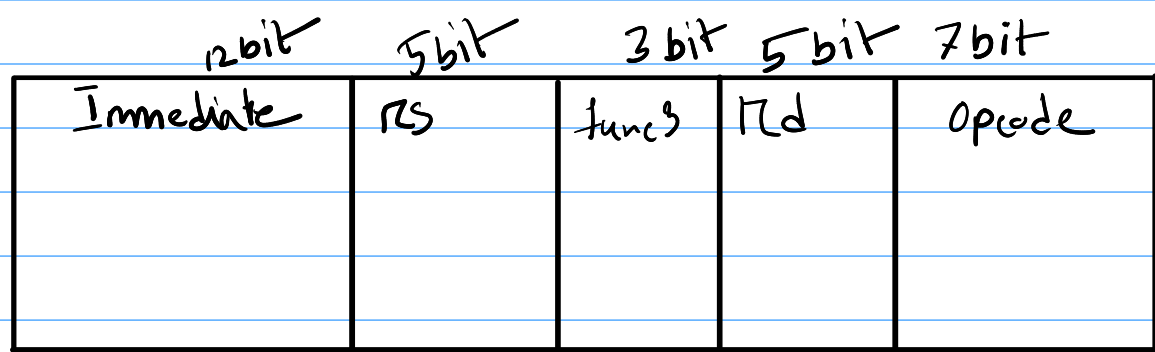
```
addi x21, x5, 5
```

7. Ld X12, 64(X16);

a) Convert the given instruction to its corresponding machine code.

b) Now from the 32 bit binary number you got in question 7.a

try to find the instruction itself again.



Construct the equivalent RISC-V code of the following C code.

Once you have the RISC-V code, identify type of each instruction and encode them accordingly.

$A[7] = A[2] + A[B[8]] + 10;$

$B[i] = A[3] - 8;$

Base addresses of array A and B are in register X20 and X21 and i is in register X22

1. LD x5, 64(x21)
slli x5,x5,3
add x18, x5, x20
LD x5, 0(x18)
LD x6, 16(x20)
add x7, x5, x6
addi x7, x7, 10
SD x7, 56(x20)

2. LD x7, 24(x20)
addi x5, x7, -8
slli x6, x22, 3
add x6, x21, x6
SD x5, 0(x6)

Step 1: $A[B[8]]$
Here $B[8]$ is a offset
now in double word number of cell $64/8 = 8$
here $B[8]$ index is 8 ; offset will be $8*8 = 64$
Address is offset + base address = $64(x21) // B = X21$

LD x5, 64(x21)
now x5 contains index for A : $A[x5]$
we have to multiply x5 with number of cell; $x5*8$
so slli x5, x5, 3

now we got the offset for A
now we have add this with the base address to get
reg address
add x18, x5, x20 // $A[B[8]]$

now we have to load this to access from the reg
LD x5, 0(x18)

$a = [(A[B[3]] + B[24] - 12) * 28] / 256$

A = X20

LD x5, 24(x21)

B = X21

slli x5, x5, 3

a = X22

add x6, x20, x5

ld x7, 0(x6)

ld x5, 192(x21)

add x18, x5, x7

addi x18, x18, -12

slli x5, x18, 5

slli x6, x18, 2

sub x18, x5, x6

srli x18, x18, 8

add x22, x18, x0

If $(i == j) :$

$$f = g + h$$

else :

$$f = g - h$$

$$g = x_{20}$$

$$h = x_{21}$$

$$f = x_{19}$$

Beq x_7, x_9, L_1

Sub x_9, x_{20}, x_2

Beq x_6, x_0, exit

$L_1:$

add x_9, x_{20}, x_1

exit:

Question - 2:

Construct the equivalent RISC-V code of the following C code.

```
for (i = 8; i > 0 ; i--) {  
    if ( A[i] == i){  
        A[2] = A [B[3]] ;  
    }  
}
```

Base addresses of array A and B are in register X_{20} and X_{21} . Also consider i is in register X_{22} .

addi $x_{22}, x_{22}, 8$ //i=8

loop:

blt x_{22}, x_0, exit //i<0

slli $x_7, x_{22}, 3$ //i*3

add x_8, x_{20}, x_7 //A[i]

ld $x_9, 0(x_8)$ //x9= A[i]

bne x_9, x_{22}, skip //A[i] != i

ld $x_5, 24(x_{21})$ //B[3]

slli $x_6, x_5, 3$ //[B[3]]

add x_{23}, x_{20}, x_6 //A[B[3]]

ld $x_{18}, 0(x_{23})$ //x18 = A[B[3]]

sd $x_{18}, 16(x_{20})$ //A[2] = A[B[3]]

beq x_0, x_0, skip

skip:

add $x_{22}, x_{22}, -1$ //i-1

beq x_0, x_0, loop

exit:

Question - 3:

Construct the equivalent RISC-V code of the following C code.

```
if ( A[i] < i){  
    A[2] = A [B[3]] ;  
}
```

Base addresses of array A and B are in register X_{20} and X_{21} . Also consider i is in register X_{22} .

loop:

```
slli x7, x22, 3          //x7= i*8  
add x8, x20,x7           //x8= A[i]  
ld x9, 0(x8)  
Blt x22, x9, exit  
ld x5, 24(x21)  
slli x5, x5, 3  
add x6, x20,x5  
ld x18, 0(x6)  
sd x18, 16(x20)  
beq x0,x0,loop  
exit
```

Question - 4:

Construct the equivalent RISC-V code of the following C code.

```
if ( A[3] != A[6]){
    if (A[3] == 0) {
        A[3] = A[3] + 2;
    }else{
        A[6] = A[6] / 16;
    }
}else{
    A[6] = A[6] * 8
}
```

Base addresses of array A and B are in register X₂₀ and X₂₁ .

```
ld x6, 24(x20)
ld x7, 48(x20)

Beq x7,x6, else1
Bne x6, x0, else2
addi x23, x6, 2
sd x23, 0(x6)
```

```
beq x0, x0, exit
```

```
else2:
srli x19, x7, 4
sd x19, 0(x7)
beq x0, x0, exit
```

```
else1:
slli x8, x7, 3
sd x7, 48(x20)
```

```
exit:
```

Q.

int number = 0x67

int b = 10

while (b <= 20):

if (number != a):

if (number > a):

number += 12

else:

number += 10

b+= 3

number -> X18

a -> X19

b -> X20

S.

addi x18, x0, 103 //x18 = 103

addi x20, x0, 10 //x20 = 10

addi x7, x0, 20 //x7 = 20

whileloop:

bge x7, x20, if_con while (20 >= b)

beq x0, x0, exit if while loop breaks

if_con:

beq x18,x19, inc if, number == a

bge x19, x18, else_con if a>= number

addi x18, x18, 12

beq x0,x0, inc

else:

addi x18,x18, 10

beq x0, x0, inc

inc:

addi x20, x20, 3

beq x0, x0, whileloop

exit:

0x67 = 103

Q.

Code - 2:

a = b + calc (4,5,6)

arr[3] = a

def calc (p, q, r):

z = p * 10

y = z + q

y += r

y = r/16

return y

Arr -> double word

Arr[0] -> x21

a -> X19

b -> X20

use arg register x12,x13,x14

z -> X19

y -> X20

S.

addi x12, x0, 4 //x12 = 4

addi x13, x0, 5 //x13 = 5

addi x14, x0, 6 //x14 = 6

Jal x1, calc

add x19, x20, x10 // a = b + y

sd x19, 24(x21) // arr[3] = a

beq x0, x0, exit

calc:

addi x2, x2, -16

sd x19, 0(x2) // Stack push

sd x20, 8(x2)

slli x5, x12, 3 // p*8

slli x6, x12, 1 // p*2

add x19, x5, x6 // z = p*8+p*2

add x20, x19, x13 // y = z+q

add x20, x20, x14 // y = y+r

srli x7, x14, 4 // x7 = r/16

sub x20, x20, x20 // y = reset

add x20, x7, x0 // y = r/16

add x10, x20, x0 //return y

ld x20, 8(x2)

ld x19, 0(x2) // restore stack

addi x2, x2, 16

Jalr x0, 0(x1)

exit:

Chapter 3

IEEE-754 Defines floating point standard

Single precision -> 32 bit

Double precision -> 64 bit

A decimal/binary number is Normalized if:

1. Only one digit before the decimal/binary point
2. And that digit must be a non-zero number. (1-9)

If you shift left exp +

If you shift right exp -

	Sign Bit	Exponent	Fraction
✓ Single P.	1 bit	8 bits	23 bits
✓ Double P.	1 "	11 "	52 "

IEEE-754 Floating Point Representation

s1: Convert it Binary

s2: Normalize the given value

s3: After normalize the value check the exponent that is the actual exponent

s4: Calculate the bias exponent; If single precision Bias = $2^{(8-1)} - 1 = 127$

if double precision Bias = $2^{(11-1)} - 1 = 1023$

s4: Now, calculate Bias Exponent = Actual + Bias

s5: Now, Convert the bias exponent into binary (If Single precision write it into 8 bits, if double precision write it into 11 bits)

s6: Sign value depend on the question if the given value is positive sign bit = 0

if the given value is negative sign bit = 1

In single precision = 32 bits

sign bit = 1 bit

exponent = 8 bits

Fraction = $(32 - 1 - 8)$ bit = 23 bits

In double precision = 64 bits

sign bit = 1 bit

exponent = 11 bits

fraction = $(64 - 1 - 11)$ bits = 52 bits

What is the work of exponent field ?

It denotes that how large and how small number you can store

For single precision

Using double precision, you can represent a larger or a smaller number than single precision.

Exponent range = $2^8 - 1 = 255$

now 0 and 255 is reserved

So, 1-254

Now, Bias in 8 bit system = $2^{(8-1)} - 1 = 127$

Now, Actual Exponent range = 1-127 to 254-127

= -126 to +127

= $2^{(-126)}$ to $2^{(127)}$

That's mean i can store minimum $Y \times 2^{(-126)}$ to maximum $Y \times 2^{(127)}$ in expo field

For Double Precision

Expo Range = $2^{11} - 1 = 2047$

Again 0 and 2047 is reserved; so range is 1 to 2046

Bias in 11 bit system = $2^{(11-1)} - 1$

= 1023

Now, Actual Exponent range = 1-1023 to 2046-1023

= -1022 to 1023

That's mean i can store minimum $Y \times 2^{(-1022)}$ to maximum $Y \times 2^{(1023)}$ in expo field

Bias.Expo > Range = Overflow

Bias.Expo < Range = Underflow

Bias Expo range = Actual + Bias

Bias = $2^{(n-1)} - 1$

Can you store a number in given IEEE format?

Check the Bias exponent is within the range or not

Can you store a number precisely in given IEEE format?

s1: Check the Bias exponent is within the range or not

s2: Check the fraction field

Question - 2:

Find the Biased Exponent of 1.1011×2^{34} in IEEE-754 single precision format.

Here, In single precision; Sign bit = 1 bit

Exponent Bit = 8 bits

Fraction = $(32-1-8) = 23$ bits

Given value is already normalized

Actual expo = 34

Bias = $2^{(8-1)} - 1 = 127$

Bias expo = $34 + 127 = 161$

(161) in Binary = 1010 0001

Question - 3:

Find the Biased Exponent of 1.1011×2^4 in 12-bit IEEE-754 format where the size of the exponent field is 4 bits.

Total size 12 bits

Sign bit = 1 bit

Exponent = 4 bit

Fraction = $(12-4-1)$ bit = 7 bits

Actual Exponent = 4

Bias = $2^{(4-1)} - 1 = 7$

Bias exponent = $7 + 4 = 11$

(11) in binary = 1011

Question - 4:

Find the Biased Exponent of 1.1011×2^{34} in 64-bit IEEE-754 format.

In 64 bit format

Sign bit = 1 bit

Exponent bit = 11 bit

Fraction bit = $(64-1-11)$ bit = 52 bits

Actual exponent = 34

Bias = $2^{(11-1)} - 1 = 1023$

Bias exponent = $1023 + 34 = 1057$

(1057) in binary = 100 0010 0001

Question - 5:

Find the Biased Exponent of 5678.898 in 34-bit IEEE-754 format where the size of the exponent field is 10 bits.

5678 in binary = 1 0110 0010 1110

.898 in binary = .111001

5678.898 = 1011000101110.111001 $\times 2^0$

In Normalize :

1.011000101110111001 $\times 2^{13}$

Actual Exponent = 13

Bias = $2^{(10-1)} - 1 = 511$

Bias Exponent = $511 + 13 = 524$

(524) in binary = 10 0000 1100

$$\begin{array}{r}
 .898 \\
 \times 2 \\
 \hline
 1 \quad .796 \\
 \times 2 \\
 \hline
 1 \quad .592 \\
 \times 2 \\
 \hline
 1 \quad .184 \\
 \times 2 \\
 \hline
 0 \quad .368 \\
 \times 2 \\
 \hline
 0 \quad .736 \\
 \times 2 \\
 \hline
 1 \quad .472 \\
 \vdots
 \end{array}$$

Question - 6:

Convert -0.0098710 in 34-bit IEEE-754 floating point representation where the size of the fraction field is 23 bits.

.0098710 in binary = .0000001010

normalize = $1.01000\dots \times 2^{-6}$

In 32 bit format

Fraction = 23 bit

sign bit = 1

Exponent = 8 bit

Bias = $2^{(8-1)} - 1 = 127$

Actual = -6

Bias Exponent = $127 - 6 = 121$

$$\begin{array}{r}
 .00987 \\
 \times 2 \\
 \hline
 0 \quad .01974 \\
 \times 2 \\
 \hline
 0 \quad .03948 \\
 \times 2 \\
 \hline
 0 \quad .07896 \\
 \times 2 \\
 \hline
 0 \quad .15792 \\
 \times 2 \\
 \hline
 0 \quad .31584 \\
 \times 2 \\
 \hline
 0 \quad .63168 \\
 \times 2 \\
 \hline
 1 \quad .26336 \\
 \times 2 \\
 \hline
 0 \quad .52672 \\
 \times 2 \\
 \hline
 1 \quad .05344 \\
 \times 2 \\
 \hline
 0 \quad .10688 \\
 \times 2 \\
 \hline
 0 \quad .21376 \\
 \times 2 \\
 \hline
 0 \quad .42752
 \end{array}$$

IEEE-754 Floating Point to Decimal Conversion

Steps:

1. Convert the Hex/Decimal number to binary number.
2. Arrange the binary number according to the given IEEE format.
3. Determine the sign.
4. Find out the actual exponent from the biased exponent field.
5. Convert Fraction to Decimal.
6. Final number = $(-1)^{\text{Sign Bit}} \times (1 + \text{Fraction}) \times 2^{\text{Actual Exponent}}$.

i. 0xFF1205BA

FF1205BA = 1111 1111 0001 0010 0000 0101 1011 1010

In single precision format

Sign bit = 1 bit

Exponent bit = 8 bit

Fraction bit = 23 bit

In here, Sign bit = 1

Exponent = 1111 1110

Fraction = 0010 0100 0000 1011 0111 010

Bias exponent = 1111 1110 = 254

Bias = $2^{(8-1)} - 1 = 127$

Actual = Bias Exponent - Bias = $254 - 127 = 127$

Fraction = $1 \times 2^{-3} + 1 \times 2^{-6} + 1 \times 2^{-13} + 1 \times 2^{-15} + 1 \times 2^{-16}$

$+ 1 \times 2^{-18} + 1 \times 2^{-19} + 1 \times 2^{-20} + 1 \times 2^{-22}$

= 0.1407998

-1.940970281x10³⁸

Floating Point Addition/Subtraction

Given A and B are both floating-point numbers.

Steps:

1. Make sure both numbers are in Binary.
2. Normalize both A and B.
3. Align the binary point so that the lower exponent matches with the higher exponent.
4. Now add or sub accordingly.
5. Normalize the result.

Question - 13:

Add the 7.01 and 0.71 using IEEE-754 single-precision floating-point representation. Check if the result has overflow or underflow.

$$7 = 111$$

$$.01 = .00000010$$

$$7.01 = 111.0000001 \times 2^0$$

$$1.110000001 \times 2^2$$

$$0.71 = .1011010111 \times 2^0$$

$$1.011010111 \times 2^{-1}$$

$$1.110000001 \times 2^2$$

$$0.001011010 \times 2^2$$

$$1.111011011 \times 2^2$$

Question - 15:

Subtract -7.01 from 18.71 using IEEE-754 single-precision floating-point representation. Check if the result has overflow or underflow.

Note: Consider 10 decimal digits while converting from decimal to binary for the following questions.

$$a = 0.0111000110 \times 2^4$$

$$b = 1.0010101101 \times 2^4$$

$$a - b = 1.001000110010 \times 2^5$$

$$\begin{array}{r} 0.0111\overset{2}{\cancel{\times}}\overset{2}{\cancel{\times}}\overset{2}{\cancel{\times}}\overset{2}{\cancel{\times}}\overset{2}{\cancel{\times}}\overset{2}{\cancel{\times}}\overset{2}{\cancel{\times}}\overset{2}{\cancel{\times}}\overset{2}{\cancel{\times}}\overset{2}{\cancel{\times}}0 \\ 1.0010101101 \\ \hline 11.0100011010 \end{array}$$

Floating Point Multiplication

Given A and B are both floating-point numbers.

Steps:

1. Make sure both numbers are in Binary.
2. Normalize both A and B.
3. Add the exponents.
4. Now multiply accordingly.
5. Normalize the result.
6. Determine the sign of the operation.

ii. 0.000101×2^{-850} and 10.1×2^{-900}

$$0.000101 \times 2^{-850}$$

$$= 1.01 \times 2^{-854}$$

$$1010.000110 \times 2^{-900}$$

$$= 1.010000110 \times 2^{-897}$$

$$1.01 \times 2^{-854} \times 1.010000110 \times 2^{-897}$$

$$= (1.01 \times 1.010000110) \times 2^{(-854-897)}$$

$$= 1.10010011110 \times 2^{(-1751)}$$

$$\begin{aligned} \text{D.Precision Bias } 2^{(11-1)}-1 \\ = 1023 \end{aligned}$$

$$\text{actual} = -1751$$

$$\begin{aligned} \text{Bias expo} &= 1023 - 1751 \\ &= -728 \end{aligned}$$

$$\begin{aligned} \text{Range} &= 2^{11} - 1 \\ &= 0 - 2046 \end{aligned}$$

// UNDERFLOW

Floating Point instructions in RISC-V

Suppose, two single prec. floating point numbers A,B are stored in memory. The memory locations are directly stored in register x_{10}, x_{11} .
Write necessary code to store the result of $A+B$ in the memory address that is stored in x_{13} .

```
flw f1, 0(x10)    //A
flw f2, 0(x11)    //B
fadd.s f3, f1,f2
fsw f3, 0(x13)
```

Question-1:

Two single precision floating point numbers a and b are stored in memory consecutively. The memory location of the first number, a is directly stored in register x_{19} .

Write necessary code that adds these two numbers and stores the result in a variable called d. Variable d is mapped to register f_{20} .

```
flw f1, 0(x19)
flw f2, 4(x19)
fadd.s f20, f1,f2
```

Question-2:

Two single-precision numbers a and b are stored consecutively in memory. The memory location of the first number, a is directly stored in register x_{19} .

Convert the given high level code to corresponding risc-v code:
 $res = (a - b)^2$

Note: res is mapped to f_{16}

```
flw f1, 0(x19)
flw f2, 4(x19)
fsub.s f3, f1,f2
fmul.s f16, f3,f3
```

Question-3:

Convert the given high level code to corresponding risc-v code:

```
if (a > b):
    a = a * d
elif (a == b):
    b -= e;
else:
    arr1[25] = b / e
```

Note: a, b, d, e variables are mapped to f_4, f_5, f_6, f_7 registers and all the numbers stored are single precision numbers. arr1 is a word array. The base address of arr1 is in register x_{20} .

```
flt.s x7, f5,f4 ;    //b<a
beq x0,x7, elifcon
    fmul.s f4, f4,f6
    beq x0,x0, exit

elifcon:
    feq.s x7,f4,f5
    beq x0,x7; elsecon
    fsub.s f5, f5,f7
    beq x0,x0,exit

elsecon:
    fdiv.s f1, f5,f7
    fsw f1,100(x20)
    exit:
```

Question-4:

Convert the given high level code to corresponding risc-v code:

if (a >= b):

 a = (a * c) + d;

else:

 b = b / c;

 arr1[25] = b ;

Note: a, b, c, d variables are mapped to f4, f5, f6, f7 registers and all the numbers stored are single precision numbers. arr1 is a word array. The base address of arr1 is in register x20.

You can not use fadd or fmul instructions here.

fle x7, f5,f4 //b<=a

beq x0,x7, elsecon

fmadd.s f4, f4,f6,f7

beq x0,x0,exit

elsecon:

fdiv.s f5, f5,f6

fsw f5, 100(x20)

exit:

Floating point instruction list:

Instruction	Syntax & Example	Operation	Note
fadd.s	fadd.s rd, rs1, rs2 Ex: fadd.s f ₃ , f ₂ , f ₅	Perform single-precision floating-point addition $rd = rs1 + rs2$	For all the red marked instructions, if the numbers are in double precision then just add "d" instead of "s"
fsub.s	fsub.s rd, rs1, rs2 Ex: fsub.s f ₃ , f ₂ , f ₅	Perform single-precision floating-point subtraction $rd = rs1 - rs2$	
fmul.s	fmul.s rd, rs1, rs2 Ex: fmul.s f ₃ , f ₂ , f ₅	Perform single-precision floating-point multiplication. $rd = rs1 * rs2$	
fdiv.s	fdiv.s rd, rs1, rs2 Ex: fdiv.s f ₃ , f ₂ , f ₅	Perform single-precision floating-point division. $rd = rs1 / rs2$	
fmadd.s	fmadd.s rd, rs1, rs2, rs3 Ex: fmadd.s f ₃ , f ₂ , f ₅ , f ₆	Perform single-precision fused multiply addition. $rd = (rs1 * rs2) + rs3$	
fmmadd.s	fmmadd.s rd, rs1, rs2, rs3 Ex: fmmadd.s f ₃ , f ₂ , f ₅ , f ₆	Perform negated single-precision fused multiply addition. $rd = -(rs1 * rs2) - rs3$	
fmsub.s	fmsub.s rd, rs1, rs2, rs3 Ex: fmsub.s f ₃ , f ₂ , f ₅ , f ₆	Perform single-precision fused multiply subtraction. $rd = (rs1 * rs2) - rs3$	
fmmsub.s	fmmsub.s rd, rs1, rs2, rs3 Ex: fmmsub.s f ₃ , f ₂ , f ₅ , f ₆	Perform negated single-precision fused multiply subtraction. $rd = -(rs1 * rs2) + rs3$	
fsqrt.s	fsqrt.s rd, rs1 Ex: fsqrt.s f ₃ , f ₂	Perform single-precision square root. $rd = \sqrt{rs1}$	
feq.s	feq.s rd, rs1, rs2 Ex: feq.s x ₃ , f ₂ , f ₅	Performs a comparison between two floating point numbers. If both are equal stores 1 in rd else stores 0.	
flt.s	flt.s rd, rs1, rs2 Ex: flt.s x ₃ , f ₂ , f ₅	Performs a comparison between two floating point numbers. If (rs1 < rs2) stores 1 in rd else stores 0.	
fle.s	fle.s rd, rs1, rs2 Ex: fle.s x ₃ , f ₂ , f ₅	Performs a comparison between two floating point numbers. If (rs1 <= rs2) stores 1 in rd else stores 0.	

Instruction	Syntax & Example	Operation
flw	flw rd, offset(rs1) Ex: flw f ₃ , 8(x ₅)	Load a single-precision floating-point value from memory into floating-point register rd.
fld	fld rd, offset(rs1) Ex: fld f ₃ , 16(x ₅)	Load a double-precision floating-point value from memory into floating-point register rd.
fsw	fsw rs2, offset(rs1) Ex: fsw f ₃ , 8(x ₅)	Store a single-precision value from floating-point register rs2 to memory.
fsd	fsd rs2, offset(rs1) Ex: fsd f ₃ , 16(x ₅)	Store a double-precision value from floating-point register rs2 to memory.
fcvt.s.w	fcvt.s.w rd, rs1 Ex: fcvt.s.w f ₃ , x ₅	Converts a 32-bit signed integer, in integer register rs1 into a floating-point number in floating-point register rd.
fcvt.w.s	fcvt.w.s rd, rs1 Ex: fcvt.w.s x ₃ , f ₅	Convert a floating-point number in floating-point register rs1 to a signed 32-bit in integer register rd.
fmv.w.x	fmv.w.x rd, rs1 Ex: fmv.w.x f ₃ , x ₅	Move the single-precision value encoded in IEEE 754 standard encoding from the lower 32 bits of integer register rs1 to the floating-point register rd.
fmv.x.w	fmv.x.w rd, rs1 Ex: fmv.x.w x ₃ , f ₅	Move the single-precision value in floating-point register rs1 represented in IEEE 754 encoding to the lower 32 bits of integer register rd.

1 `fcvt.s.w f3, x5` → Int → Float

- Imagine you have an **integer** in `x5`, e.g., `7`.
- Your CPU wants it as a **floating-point number** (like 7.0 in IEEE-754 format).
- `fcvt.s.w` **converts it**: it takes the numeric value `7` and turns it into a floating-point representation `0x40E00000`.

Think: "I have a normal number, I want it as a float."

2 `fcvt.w.s x3, f5` → Float → Int

- Imagine you have a **float** in `f5`, e.g., `3.14`.
- You want an integer version.
- `fcvt.w.s` **takes the value and converts it to integer**, so `x3 = 3`.

Think: "I have a float, I want the closest integer."

3 `fmv.w.x f3, x5` → Move bits Int → Float

- You have an **integer** in `x5`, e.g., `0x40400000`.
- You **don't care about its numeric value**; you just want the CPU to treat the **same 32 bits** as a float.
- `f3` now contains `0x40400000` as a float → interpreted as `3.0`.

Think: "Copy the bits exactly, no conversion, just reinterpret them as float."

4 `fmv.x.w x3, f5` → Move bits Float → Int

- You have a **float** in `f5`, e.g., `3.0` → IEEE-754 bits `0x40400000`.
- You want the **same bits in an integer register**.
- `x3` now contains `0x40400000` (decimal 1077936128).
- **Notice**: the numeric value changes because you're looking at bits as integer now.

Think: "Copy the bits exactly, don't convert the number."

Long Multiplication

Example:

Multiply 8 and 9 using the long multiplication method:

Solution:

You can choose any of the operands as multiplier or multiplicand.

Number of iterations = Number of bits in Multiplier

Iteration	Multiplier	Multiplicand	Product
0	1001	0000 1000	0000 0000
1	1001	0000 1000	0000 1000
	1001	0001 0000	0000 1000
	0100	0001 0000	0000 1000
2	0100	0010 0000	0000 1000
	0010	0010 0000	0000 1000
3	0010	0100 0000	0000 1000
	0001	0100 0000	0000 1000
4	0001	0100 0000	0100 1000
	0001	1000 0000	0100 1000
	0000	1000 0000	0100 1000

If LSB of multiplier = 1

s1: Multiplicand + product

s2: Right shift Multiplier

s3: Left shift Multiplicand

If LSB of multiplier = 0

s1: Right shift Multiplier

s2: Left shift Multiplicand

Number of iterations =

Number of bits in Multiplier

Ex:

10 = 1010

9 = 1001 Num of iteration = Number of bits in multiplier

Iteration	Multiplier	multiplicand	product
0th	1010	0000 1001	0000 0000
1st	0101	0000 1001	0000 0000
	0101	0001 0010	0000 0000
2nd	0101	0001 0010	0001 0010
	0010	0001 0010	0001 0010
	0010	0010 0100	0001 0010
3rd	0001	0010 0100	0001 0010
	0001	0100 1000	0001 0010
4th	0001	0100 1000	0101 1010
	0000	0100 1000	0101 1010
	0000	1001 0000	0101 1010

Optimized Multiplication

Number of iterations =
Number of bits in Multiplier

Example:

Multiply 8 and 9 using the optimized multiplication method:

Solution:

You can choose any of the operands as multiplier or multiplicand.

Number of iterations = Number of bits in Multiplier

This color represents the product MSB part.

Multiplier = Product LSB 4 bits

Iteration	Multiplicand	Product
0	1000	0000 1001
1	1000	1000 1001
		0100 0100
2	1000	0010 0010
3	1000	0001 0001
4	1000	1001 0001
		0100 1000

If product LSB = 0;

s1: Product Right shift

If product LSB = 1;

s1: Multiplicand + Product
msb 4 bits

s2: Right Shift product

Ex:

10 = 1010

9 = 1001 Num of iteration = Number of bits in multiplier

Iteration	Multiplicand	Product
0th	1010	0000 1001
1st	1010	1010 1001
	1010	0101 0100
2nd	1010	0010 1010
3rd	1010	0001 0101
4th	1010	1011 0101
	1010	0101 1010

Ex: 1.1011×2^{34} ; Find the Biased Exponent of the given number in IEEE 754 single precision format.

Solⁿ: Check if The number is in normalized format or not.

↳ If no. then normalize it and proceed.

Sd $1.1011 \times 2^{34} \rightarrow$ It is in normalized form

For Single Precision $\Rightarrow 2^{n-1} - 1$

$$\Rightarrow 2^{8-1} - 1 \Rightarrow 127$$

Bias Exponent $\Rightarrow$ Actual + Bias

$$\Rightarrow 34 + 127 \Rightarrow 158$$

For single precision $\Rightarrow$	1 bit	8 bit	23 bit
	Sign	Exponent	Fraction
	0	1001 110	1011 0000 0000 0000 0000 000

Ex: 11.1011×2^{-8} ; Find the Biased Exponent of the given number in IEEE 754 single precision format.

11.1011×2^{-8}
 $\Rightarrow$ For Normalized form

$$1.11011 \times 2^{-8+1} \rightarrow 1.11011 \times 2^{-7}$$

$$\text{Bias} \Rightarrow 2^{n-1} - 1$$

$$\Rightarrow 2^{8-1} - 1 \Rightarrow 127$$

$$\text{Bias Expo} \Rightarrow 127 + (-7)$$

$$\Rightarrow 120$$

$$\Rightarrow 0111\ 1000$$

1 bit Sign bit	8 bit Exponent	23 bit Fraction
0	0111 1000	1101 1000 0000 0000 0000 000

Ex: 11.1011×2^{-8} ; Find the Biased Exponent of the given number in 15 bit IEEE

format where size of the fraction field is 8 bits.

$$1.11011 \times 2^{-7}$$

Soln:

$$15 - 7 = 8$$

1 bit Sign bit	6 bit Exponent	8 bit Fraction
0	01 1000	1101 1000

$$\text{Bias} \Rightarrow 2^{b-1} - 1$$

$$\Rightarrow 31$$

$$\text{Bias Expo} \Rightarrow \text{Actual} + \text{Bias}$$

$$\Rightarrow -7 + 31$$

$$\Rightarrow 24$$

Decimal to Floating Point Conversion:

- (i) Convert the decimal numbers to binary numbers.
- (ii) Normalize the binary numbers.
- (iii) Find the biased exponent.
- (iv) Sign Bit
- (v) Find the fraction.

Ex: Convert 50.6749 to 32 bit IEEE-754 Floating point representation :-

$50 \Rightarrow (110010)_2$ $.6749 \Rightarrow .1010110011\dots$ $110010.1010110011\dots \times 2^0$ $1.100101010110011\dots \times 2^5$	<table border="0"> <tr><td>→</td><td>1</td><td>.6749</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>1</td><td>.3498</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>0</td><td>.6996</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>1</td><td>.3992</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>0</td><td>.7984</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>1</td><td>.5968</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>1</td><td>.1936</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>0</td><td>.3872</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>0</td><td>.7744</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>1</td><td>.5568</td></tr> <tr><td></td><td></td><td>× 2</td></tr> <tr><td></td><td>1</td><td>.1136</td></tr> </table>	→	1	.6749			× 2		1	.3498			× 2		0	.6996			× 2		1	.3992			× 2		0	.7984			× 2		1	.5968			× 2		1	.1936			× 2		0	.3872			× 2		0	.7744			× 2		1	.5568			× 2		1	.1136	$\text{Bias} \Rightarrow 2^{8-1} - 1$ $\Rightarrow 2^7 - 1 \Rightarrow 127$ $\text{Bias Expo} \Rightarrow \text{Actual} + 127$ $\Rightarrow 5 + 127 \Rightarrow 132$ $132 \Rightarrow 1000\ 0100$
→	1	.6749																																																															
		× 2																																																															
	1	.3498																																																															
		× 2																																																															
	0	.6996																																																															
		× 2																																																															
	1	.3992																																																															
		× 2																																																															
	0	.7984																																																															
		× 2																																																															
	1	.5968																																																															
		× 2																																																															
	1	.1936																																																															
		× 2																																																															
	0	.3872																																																															
		× 2																																																															
	0	.7744																																																															
		× 2																																																															
	1	.5568																																																															
		× 2																																																															
	1	.1136																																																															

Sign	Expo	Fraction
0	1000 0100	01001 0101 0110 0110 0000 0000 0000

Hex: $0x9AB3000$

Convert -0.0232 to 12 bit IEEE-754 floating point representation,
 where biased exponent is 4 bits.

Dig
 $2^{41} - 1$
 $\Rightarrow 7$

$$0.0000010111 \dots \times 2^0$$

$$1.0111 \dots \times 2^{-6}$$

Bias Expo
 $\Rightarrow -6 + 7$
 $\Rightarrow 1$

Sign	Expo	fraction
1	0001	011 000

$$\begin{array}{r}
 0.0232 \\
 \times 2 \\
 \hline
 0 \quad 0464 \\
 \times 2 \\
 \hline
 0 \quad 0928 \\
 \times 2 \\
 \hline
 0 \quad 1856 \\
 \times 2 \\
 \hline
 0 \quad 3712 \\
 \times 2 \\
 \hline
 0 \quad 7424 \\
 \times 2 \\
 \hline
 1 \quad 4848 \\
 \times 2 \\
 \hline
 0 \quad 9696 \\
 \times 2 \\
 \hline
 1 \quad 9392 \\
 \times 2 \\
 \hline
 1 \quad 8784 \\
 \times 2 \\
 \hline
 1 \quad 7568
 \end{array}$$

Floating Point to Decimal:

(i) Hex to Binary.

(ii) Arrange the binary according to the format.

(iii) Determine the sign

(iv) Find out the exponent from biased exponent.

(v) Convert fraction to decimal.

$$(vi) \text{ Decimal Number} = (-1)^{\text{sign bit}} \times (1 + \text{Fraction}) \times 2^{(\text{Exponent})}$$

Example: 0xF2400120; convert this single precision floating point number to decimal.

F 2 4 0 0 1 2 0
 111 0010 0100 0000 0000 0001 0010 0000

Sign	Exponent	Fraction
1	1110 0100	1000 0000 0000 0010 0100 000

$$\text{Bias} \Rightarrow 2^{n-1} - 1 \Rightarrow 2^{8-1} - 1 \Rightarrow 127$$

$$\text{Bias Exponent} \Rightarrow 1110 \ 0100 \Rightarrow 228$$

$$\text{Actual Expo} \Rightarrow 228 - 127 \Rightarrow 101$$

Fraction 1000 0000 0000 0010 0100 000

$$\begin{aligned} \Rightarrow & 1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} + 0 \times 2^{-4} + 0 \times 2^{-5} + 0 \times 2^{-6} + 0 \times 2^{-7} + 0 \times 2^{-8} + 0 \times 2^{-9} + 0 \times 2^{-10} + 0 \times 2^{-11} + 0 \times 2^{-12} + 0 \times 2^{-13} \\ & + 0 \times 2^{-14} + 1 \times 2^{-15} + 0 \times 2^{-16} + 0 \times 2^{-17} + 1 \times 2^{-18} + 0 \times 2^{-19} + 0 \times 2^{-20} + 0 \times 2^{-21} + 0 \times 2^{-22} + 0 \times 2^{-23} \\ \Rightarrow & 0.5000343323 \end{aligned}$$

$$(-1)^1 \times (1 + 0.5000343323) \times 2^{101}$$

$$\Rightarrow -3.80303843 \times 10^{30}$$

Reverse

$$-3.80303843 \times 10^{30}$$

Question - 1:

Normalize the following numbers:

	Given Number	Normalized Number
i.	0.0000124678_{10}	1.24678×10^{-5}
ii.	$1584.234_{10} \times 10^5$	1.584234×10^8
iii.	4782.2354_{10}	4.7822354×10^3
iv.	1101011111_2	1.10101111×2^5
v.	0.001100_2	1.100×2^{-3}
vi.	$1101.1111_2 \times 2^5$	1.101111×2^8

Question - 2:

Find the Biased Exponent of 1.1011×2^{34} in IEEE-754 single precision format.

Question - 3:

Find the Biased Exponent of 1.1011×2^4 in 12-bit IEEE-754 format where the size of the exponent field is 4 bits.

Q₂

$$\text{Bias} \Rightarrow 2^{8-1} - 1 = 2^7 - 1 \Rightarrow 127$$

$$\text{Bias expo} \Rightarrow 34 + 127 \Rightarrow (161)_{10} \Rightarrow 10100001$$

Q₃

$$\text{Sign bit} = 1, \text{Exponent} = 4, \text{fraction} = 7 \text{ bit}$$

$$\text{Bias} \Rightarrow 2^{4-1} - 1 = 2^3 - 1 = 7$$

$$\text{Bias Expo} \Rightarrow 4 + 7 \Rightarrow (11)_{10} \Rightarrow (1011)_2$$

Bias $\Rightarrow 2^{11-1} - 1 \Rightarrow 1023$
 Actual $\Rightarrow 34$
 Bias Expo $\Rightarrow (1057)_{10} \Rightarrow 100\ 0010\ 0001$

Find the Biased Exponent of 1.1011×2^{34} in 64-bit IEEE-754 format.

Question - 5: Sign = 1 | fraction = $34 - 11$
 Expo = 10 $\Rightarrow 23$

Find the Biased Exponent of 5678.898 in 34-bit IEEE-754 format where the size of the exponent field is 10 bits.

Question - 6:

Convert -0.00987_{10} in 34-bit IEEE-754 floating point representation where the size of the fraction field is 23 bits.

$$1 + 10 + 23 = 34$$

sign bit 1	exponent 10	fraction 23
------------	-------------	-------------

Question - 7:

Convert $1101.1111_2 \times 2^5$ in 32-bit IEEE-754 floating point representation.

Question - 8:

Convert $1101.1111_2 \times 2^{212}$ in 64-bit IEEE-754 floating point representation. Consider 10 decimal digits when you are converting from decimal to binary.

Question - 9:

Convert the following IEEE-754 single-precision floating point numbers into decimal.

	Given Numbers	Decimal Representation
i.	0xFF1205BA	
ii.	3457890989_{10}	
iii.	23245613451_8	

Q5 5678.898

$$5678 \Rightarrow 101100010110.11100... \times 2^0$$

$$\Rightarrow 1.0110001011011100... \times 2^{12}$$

$$\text{Bias} \Rightarrow 2^{10-1} - 1 \Rightarrow 2^9 - 1 \Rightarrow 511$$

$$\text{Actual} \Rightarrow 12 \therefore \text{Bias Expo} \Rightarrow 511 + 12 \Rightarrow 523$$

$$(1000001011)_2$$

	.898
	$\times 2$
1	796
	$\times 2$
1	592
	$\times 2$
1	184
	$\times 2$
0	368
	$\times 2$
0	736

Q6 -0.00987

$$0.000000101 \times 2^0$$

$$\Rightarrow 1.01 \times 2^7$$

$$\text{Expo} \Rightarrow 2^{10-1} - 1 \Rightarrow 511$$

$$\text{Bias Expo} \Rightarrow 511 + 7 \Rightarrow 518$$

	.00987
	$\times 2$
0	001974
	$\times 2$
0	03948
	$\times 2$
0	07896
	$\times 2$
0	15792
	$\times 2$
0	31584
	$\times 2$
0	63168
	$\times 2$
1	26336
	$\times 2$
0	52672
	$\times 2$
1	05344

$\underline{\underline{Q7}} \quad 11011111 \times 2^5$ $\Rightarrow 1.1011111 \times 2^8$	$\text{BIAS} \Rightarrow 2^{n-1} - 1$ $\Rightarrow 127$ $\text{Bias Expo} \Rightarrow 127 + 8$ $\Rightarrow 135$
---	---

$\underline{\underline{Q8}} \quad 11011111 \times 2^{12}$ 1.1011111×2^{15}	$\text{Bias} \Rightarrow 2^{11-1} - 1 = 2^{10} - 1$ $\Rightarrow 1023$
---	--

Bias Expo $\Rightarrow 1238 \Rightarrow$ 100110010

Sign	
0	$1001101010 \quad 10111110 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000$ $0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000 \quad 0000$

0x9D6BF 00000000000000

You are asked to store the binary number -1.1×2^{-30} in an IEEE format. What is the minimum number of bits you need to reserve for the exponent field?	2
You are given the following IEEE representation where 7 bits are used to store the exponent. Convert 0xABCD to a binary normalized floating point number.	3

9) a) $Bias = 2^{n-1} - 1$

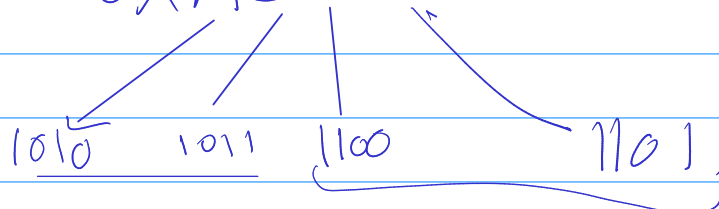
$\Rightarrow 2^5 - 1 = 31$

$Bias_{Expo} = -30 + 31$

$\Rightarrow 1$ $\therefore$ minimum 6 bit need for exponent field.

b)

0xABCD



Sign bit	Exponent	Fraction
1	010 1011	1106 1161

43

$2^{7-1} - 1$

$\Rightarrow 63$

Actual Expo = 63 - 43

$\Rightarrow 20$

$\Rightarrow (-1)^1 \times (1 + 0.80078125) \times 2^{20}$

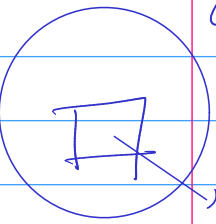
$\Rightarrow -1.80078125 \times 2^{20}$

$\Rightarrow -1.888256 \times 10^6$

Chapter 1

IR Calculation

Wafer Area $\Rightarrow \pi r^2$



Die area $\Rightarrow$ Height $\times$ Width
Die

Die per wafer = $\frac{\text{Wafer Area}}{\text{Die area}}$

Cost per die $\Rightarrow \frac{\text{Cost per wafer}}{\text{Die per wafer} \times \text{yield}}$

Yield $\Rightarrow \frac{1}{\left(1 + (\text{Defect per area} \times \text{Die area})\right)^n}$

Working Dies $\Rightarrow$ Dies per wafer $\times$ yield

Question - 1:

Assume a 19 cm diameter wafer, X has a cost of 20, contains 89 dies, and has 0.023 defects/cm². Assume another 20 cm diameter wafer, Y has a cost of 15, contains 100 dies, and has 0.031 defects/cm². Consider N = 2.

- A. What do you understand by yield?
- B. Dies per wafer $\approx$ Wafer area / Die Area - Explain why there is " $\approx$ " not " $=$ "
- C. Find the yield for both wafers.
- D. Find the cost per die for both wafers.

$$e) \quad r \Rightarrow \frac{19}{2} \Rightarrow 9.5$$

$$\text{wafer area} \Rightarrow \pi \times (9.5)^2 \\ \Rightarrow 283.52$$

$$\text{Die area} \Rightarrow \frac{283.52}{89} \Rightarrow 3.19$$

$$\text{Yield} = \frac{1}{\left(1 + (\text{Defect} \times \text{Die area})\right)^N} \\ \Rightarrow \frac{1}{\left(1 + (0.023 \times 3.19)\right)^2} \\ \Rightarrow 0.86$$

$$\text{Yield Y} \Rightarrow \frac{1}{\left(1 + (\text{Defect} \times \text{Die area})\right)^N} \\ \Rightarrow \frac{1}{\left(1 + (0.031 \times 3.14)\right)^2} \\ \Rightarrow 0.83$$

$$\text{Die area} \\ \Rightarrow \frac{314.15}{100} \\ \Rightarrow 3.14$$

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{total die per wafer}}$$

$$\Rightarrow \frac{20}{80 \times 0.86}$$

$$\Rightarrow 0.26$$

$$\text{Cost per die} \Rightarrow \frac{15}{100 \times 0.83}$$

$$\Rightarrow 0.18$$

Question 2:

Suppose gaming consoles PlayStation 5 and Xbox Series X use different implementations of AMD's Zen 2 architecture. The instructions they support can be divided into four classes according to their CPI (class A, B, C, and D). PlayStation has a clock rate of 2.7 GHz and the instruction classes have CPIs of 7, 2, 3, and 6 respectively whereas Xbox has a clock rate of 3.0 GHz and the instruction classes have CPIs of 5, 4, 2, and 1 respectively. Now suppose, a program has an instruction count of 1.0×10^6 , and the instructions are divided into classes as follows:

- 30% class A,
- 50% class B,
- 10% class C, and

10% class D.

Now answer the following questions:

1. Calculate how many more clock cycles per instruction on average does the PlayStation take compared to the Xbox?
2. Calculate the difference between the execution time in these two consoles in milliseconds
3. If the program runs on a reference PC with an execution time of 120 ms, calculate the SPECratio for the PlayStation

Execution time $\Rightarrow$ $\frac{\text{Inst Count} \times \text{Clock cycle}}{\text{Clock cycle time}}$

Class A $\Rightarrow$	3×10^5	7	5
Class B $\Rightarrow$	5×10^5	2	4
Class C $\Rightarrow$	1×10^5	3	2
Class D $\Rightarrow$	1×10^5	6	1

$$\text{Total clock cycle} \Rightarrow (3 \times 10^5 \times 7) + (5 \times 10^5 \times 2) + (1 \times 10^5 \times 3) + (1 \times 10^5 \times 6)$$

$$\Rightarrow 9 \times 10^6 \quad \text{Avg CPI} \Rightarrow \frac{9 \times 10^6}{1 \times 10^6} = 9$$

$$\text{Total cycle} \Rightarrow 3.8 \times 10^6$$

$$\text{Diff} = (9 - 3.8) \times 10^6 \Rightarrow 0.2 \times 10^6$$

$$\text{Execution time} \Rightarrow \text{Ins count} \times \text{Avg CPI} \times \text{Clock cycle time}$$

$$\Rightarrow 1 \times 10^6 \times 9 \times 10^6 \times \frac{1}{2.7 \times 10^9}$$

$$\Rightarrow 1481.48 \text{ ms}$$

$$\text{Execution time} \Rightarrow 1 \times 10^6 \times 3.8 \times 10^6 \times \frac{1}{3 \times 10^9}$$

$$\approx 1266.66 \text{ ms}$$

$$\text{Difference} \Rightarrow 214.82 \text{ ms}$$

$$\text{Speed Ratio} \Rightarrow \frac{\text{Ref time}}{\text{Execution time}}$$

$$\Rightarrow \frac{120}{1481.48}$$

$$\Rightarrow 0.081$$

Question - 3:

Consider three different processors P1, P2, and P3 executing the exact same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.

Now answer the following questions:

- Which processor has the highest performance expressed in instructions per second?
- If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.
- We are trying to reduce the execution time by 30%, but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?

P1A

$$\text{Execution time} = I \times 1.5 \times \frac{1}{3 \times 10^9}$$
$$\Rightarrow I \times 5 \times 10^{-10}$$

$$\text{P2B}, \Rightarrow I \times 4 \times 10^{-10}$$

$$\text{P3} \Rightarrow I \times 5.5 \times 10^{-10}$$

Let's take Inst count, $I = 1$ as same ISA

$$\text{Per A} = \frac{1}{5 \times 10^{-5}} = 2 \times 10^9$$

$$\text{Per B} \Rightarrow \frac{1}{4 \times 10^{-10}} \Rightarrow 2.5 \times 10^9$$

$$\text{Per C} \Rightarrow \frac{1}{5.5 \times 10^{-10}} \Rightarrow 1.8 \times 10^9$$

$\therefore$ B is more powerful

$$b) \text{Execution time} = \text{Inst count} \times \text{Avg CPI} \times \text{Clock cycle time}$$

$$10 \times 10^3 \Rightarrow \text{Inst count} \times 1.5 \times \frac{1}{3 \times 10^9}$$

$$\text{Inst count} \Rightarrow \frac{10 \times 3 \times 10^9}{1.5} \Rightarrow 2 \times 10^{10}$$

$$\text{Inst count} \Rightarrow 10 \times 2.5 \times 10^9 \Rightarrow 2.5 \times 10^{10}$$

$$\text{Inst count} \Rightarrow \frac{10 \times 4 \times 10^9}{2.2} = 1.8 \times 10^{10}$$

$$\text{Total cycle} \Rightarrow \text{Clock per Inst} \times \text{Inst Count}$$

$$\Rightarrow 3 \times 10^{10}$$

$$\Rightarrow 2.5 \times 10^{10}$$

$$\Rightarrow 3.94 \times 10^{10}$$

$$c) \text{Execution time} \Rightarrow 7 \quad \text{CPI} (1.8, 1.2, 2.64)$$

$$\text{Exec} = \text{Inst Count} \times$$

Question - 4:

Consider a computer running a program that requires 250s, with 70s spent executing add instructions, 85s executed sub instructions, and 40s spent executing left-shift instructions.

- By how much is the total time reduced if the time for add operations is reduced by 20%?
- Can the total time can be reduced by 20% by reducing only the time for left-shift instructions?

a)

$$56 + 85 + 40 = 181$$
$$\Rightarrow 69$$

$\Rightarrow$ 181

$p = C \times V + F$

~~$0.85 \times (0.814) + F$~~

~~$p + F + F$~~

$\Rightarrow 0.85 \times (0.81)$

$\Rightarrow 0.6885$

- b) Suppose a program is running on a PC with 3.2Ghz AMD Ryzen 5 processor. The program consists of 3 major types of instructions. Their instruction counts and CPI are given below:

Instruction Type	Instruction Count($\times 10^9$)	CPI
Load	3	2.3
Sub	5	?
Add	2	2.0

If the average CPI of the program is 6.3 then **what** is the CPI of the Sub Instruction?

3

- c) Based on the results of the SPEC CPU2000 benchmark conducted on an Intel Core i5 13th Gen processor, the system has an average CPI of 2.4 and an instruction count of 5×10^8 . The system completes 5,4000 clock cycles in 12 seconds, and the reference time is 9,650 seconds.

Find the SPEC ratio. Based on your findings, make a comment on the performance of the processor.

3

Execution $2.4 \times 5 \times 10^8 \times 2.22 \times 10^{-9}$

Total Ins Count $\Rightarrow (3+5+2) \times 10^9$
 $\Rightarrow 10 \times 10^9$

$$6.3 = \frac{(3 \times 10^9 \times 2.3) + (5 \times 10^9 \times x) + (2 \times 10^9 \times 2)}{10 \times 10^9}$$

$$\Rightarrow 10.42$$

* Two types of Risc-V implementations -

(i) A simplified version (Slow, too much time wasted)

(ii) A more realistic pipelined version (Optimized)

* We will only look into subset of instructions divided into 3 categories -

(i) Memory Reference: ld, sd → (main memory associated)

(ii) Arithmetic / Logical: add, sub, and, or

(iii) Control Transfer: beq

→ (Branching)

Instruction Execution

1. Instruction Memory: we only store the instructions
2. Data Memory : We store the actual Data

Program Counter holds the address location of the instruction;

Step 1 (Common): Take the address and go to the instruction memory and fetch the instruction. (#8000 → add x20, x11, x16)

Step 2 (Common) : The fetched ins will now be decoded.

Step 3: Depending Instruction class

Use ALU to calculate

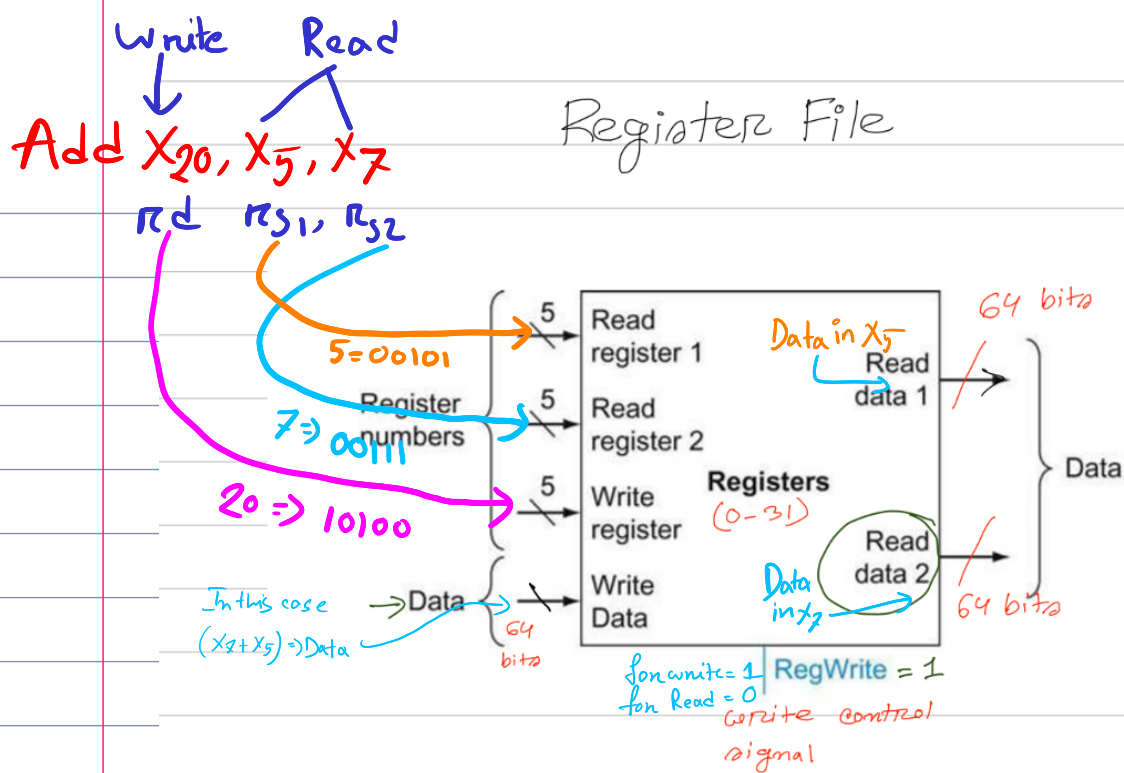
Arithmetic result

Memory address for load/store

Branch comparison

Access data memory for load/store

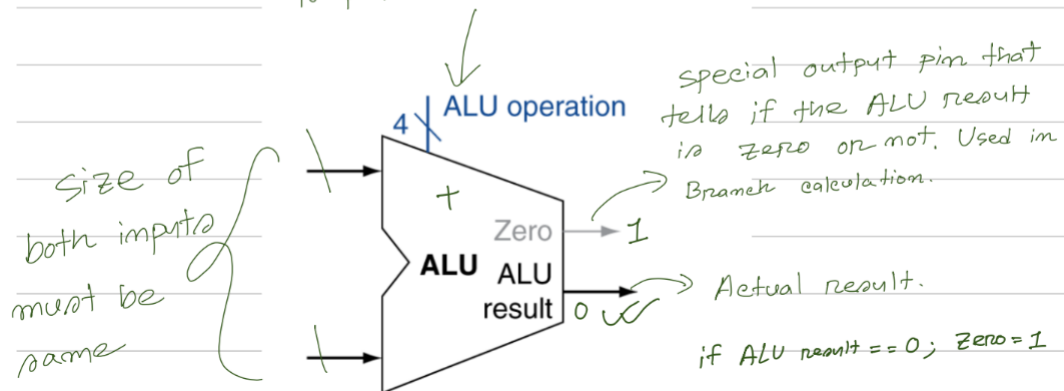
Step 4 : PC+4



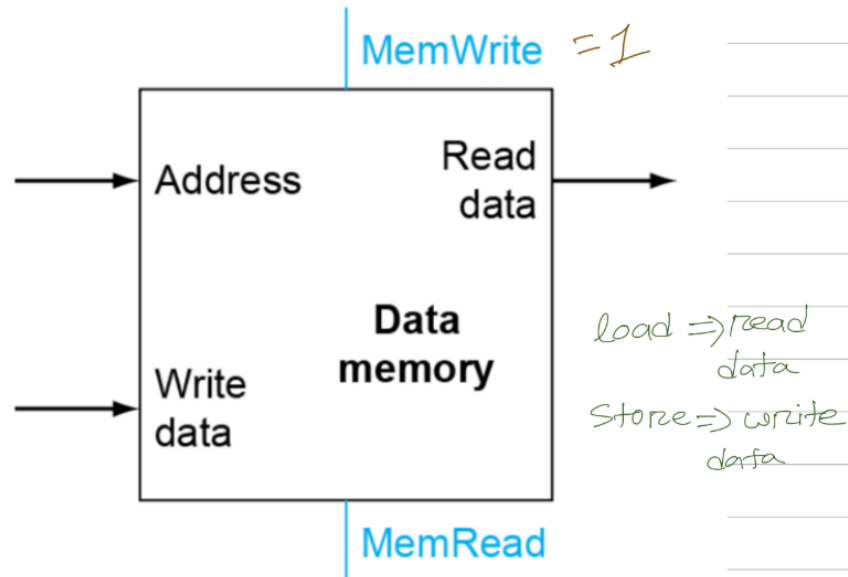
ALU Operation

ALU (Arithmetic Logic Unit)

tells ALU which operation to perform on the inputs.

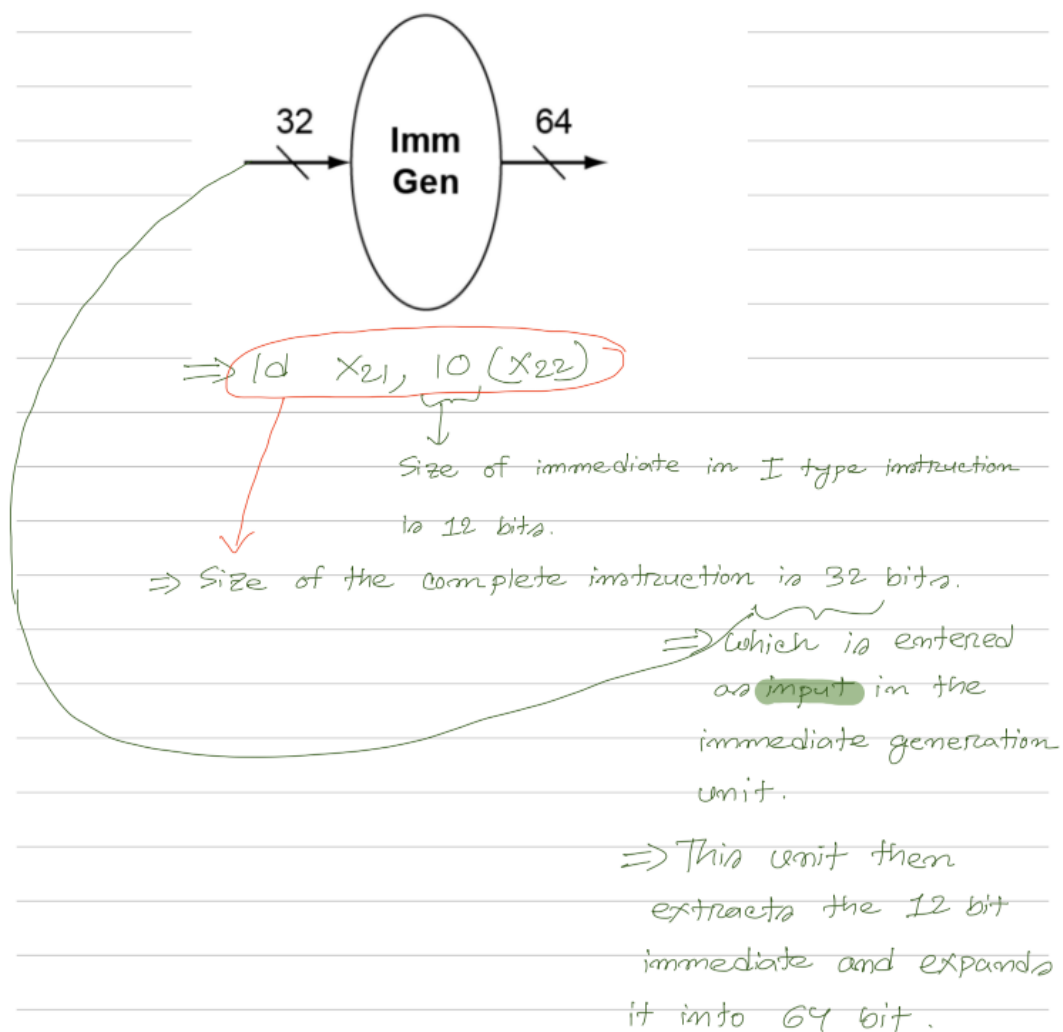


Data Memory



Immediate Generation unit:

$\rightarrow$ Expands 12 bit immediate to 64 bit value.



R type = Dont use data memory but use the reg file

LD/SD = Use both data mem and reg file

Load

LD X20, 16(X7)
Base offset $\Rightarrow X7 + 16$

Base add

ALU
Operation

64bit

~~45bit~~

Zero

ALU

ALU
Result

32bit

Imm
Gen

64bit

offset

64
bit

Immediate
32 bit
machine
code

Mem write = 0

Address

Data
Memory

Write Data

Read Data

Reg File Write
pin

Mem Read = 1

Store

store/write/copy

SD

$x_{20}, 16(x_7)$

Base to offset $\Rightarrow x_7 + 16$

Base add

ALU
Operation

64bit

~~4bit~~

Zero

ALU

ALU
Result

32bit

Imm
Gen

64bit

offset

64
bit

Immediate
32 bit
machine
code

Mem write = 1

Address

Data

Memory

Write Data

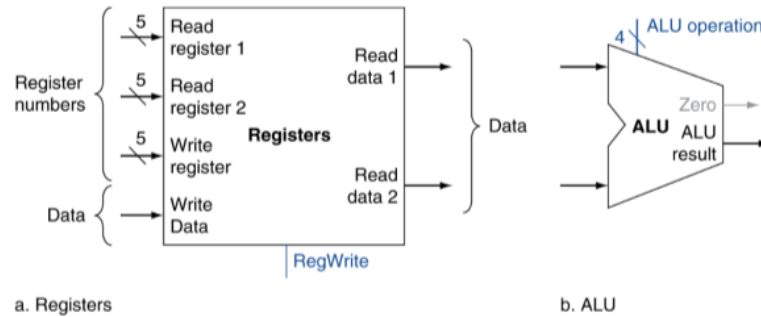
Read Data

Reg File Write
pin

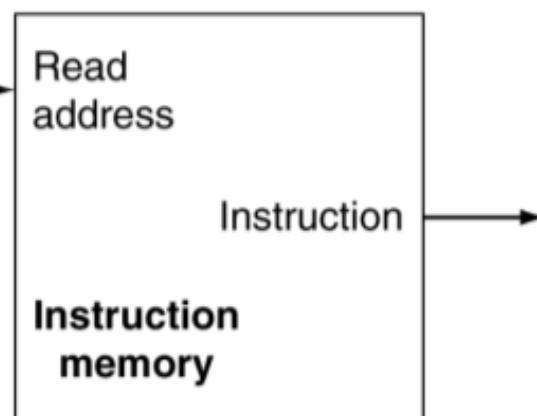
Mem Read = 0

R-Format Instructions

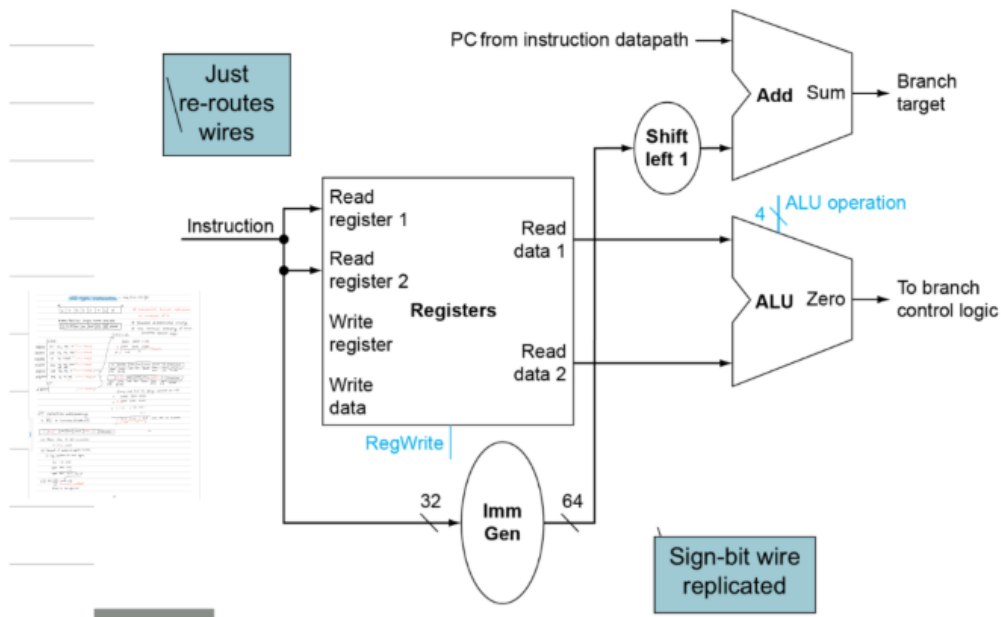
- Read two register operands
- Perform arithmetic/logical operation
- Write register result



Instruction Memory



Branch Instructions

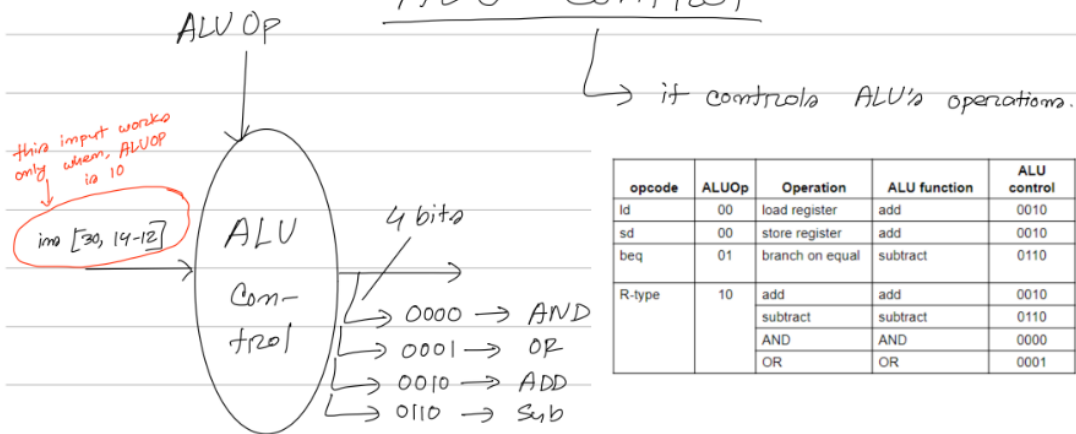


Control Unit



ALU Op $\Rightarrow$ 00 $\rightarrow$ load / store / Addi
 01 $\rightarrow$ Beq
 10 $\rightarrow$ Rtype

ALU Control



opcode	ALUOp	Operation	ALU function	ALU control
ld	00	load register	add	0010
sd	00	store register	add	0010
beq	01	branch on equal	subtract	0110
R-type	10	add	add	0010
		subtract	subtract	0110
		AND	AND	0000
		OR	OR	0001

Pipeline

⇒ Typically Load instruction takes the longest time to execute

Sub = 8 ps
Mul = 10 ps
la = 15 ps
sw = 13 ps

Clock period = 15 ps
∴ Total time to execute this sequence = 15 + 15 + 15 + 15 = 60 ps

∴ Actual time to execute this sequence = 8 + 10 + 15 + 13 = 46 ps

Time Wastage = (60 - 46) ps = 14 ps

to overcome this wastage we move on to
pipelining.
(parallel execution of instruction)

RISC-V Pipeline has 5 stages

1. Instruction fetch
2. Instruction decode/Read data
3. Execution/ ALU operation
4. Memory access(Read/write from memory)
5. Write (Write to register)

Clock Period

In single cycle clock period will be the longest instruction time

In pipeline clock period will be the longest stage time

Calculation of clock cycle

Step 1: Figure out what is happening in the instruction

step 2: calculate the total time to complete the instruction

step 3: Determine the clock period

In single cycle clock period will be the longest instruction time

In pipeline clock period will be the longest stage time

step 4: Total time/duration = clock cycle period x number of clock cycle

R-Type : IF-→ ID-→ Execution-→ Register write

LD : IF-→ ID-→ Execution-→ Mem Access-→ Register write

SD : IF-→ ID-→ Execution -→ Mem access

BEQ : IF-→ ID-→ Execution

Structural Hazard

If we try to access the same component multiple time in single clock cycle
then structural hazard occurs

That's why instruction memory and data memory are separate in datapath if we merge instruction memory and data memory then after 3 instructions at 4th instruction structural hazard will occur.

Ins Mem } merged
Data Mem } together

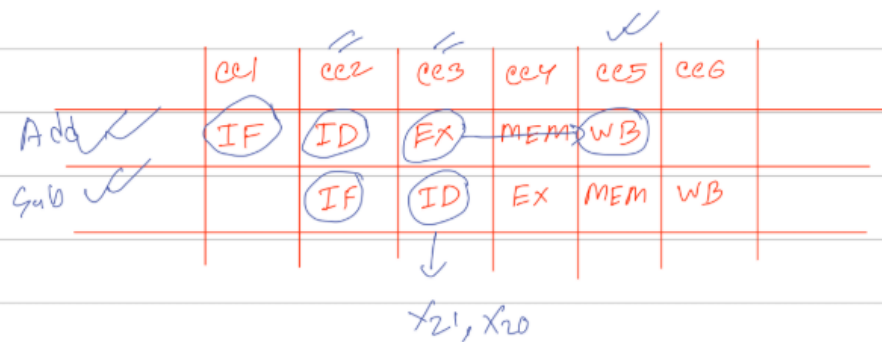
Structural Hazard

	cc1	cc2	cc3	cc4	cc5	cc6	cc7	cc8	cc9	cc10
(1)	If	ID	Ex	MEM	WB					
(2)		If	ID	Ex	MEM	WB				
(3)			If	ID	Ex	MEM	WB			
(4)				If	ID	Ex	MEM	WB		
(5)					If	ID	Ex	MEM	WB	
(6)						If	ID	Ex	MEM	WB

Data Hazard

Data Hazard

Add x_{21}, x_{22}, x_{23} ✓
Sub x_{24}, x_{21}, x_{20} ✓



If the current instruction line operand, updated in previous instruction line then the data hazard will occurred

Ex;

ADD x_{21}, x_{22}, x_{23}

SUB x_{24}, x_{21}, x_{20}

Here, x_{21} was update in ADD operation but due to pipeline method both instruction has been fetched and also decoded so x_{21} value wont be updated in the SUB operation. Hence, Data hazard occurred.

Control Hazard

A control hazard happens in a pipeline when the CPU doesn't yet know which instruction to fetch next because of a branch or jump instruction.

In a classic 5-stage pipeline:

IF – fetch instruction

ID – decode instruction

EX – execute (calculate branch target, compare registers, etc.)

MEM – memory access

WB – write back

☞ The problem:

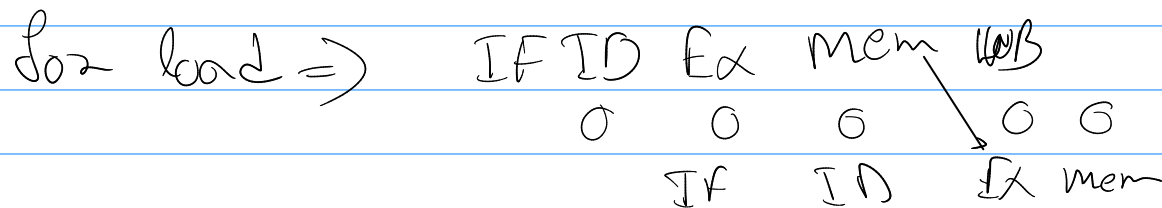
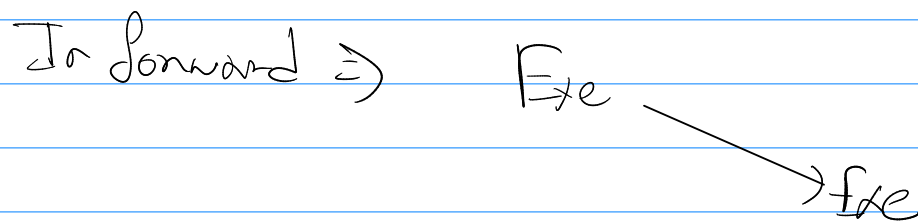
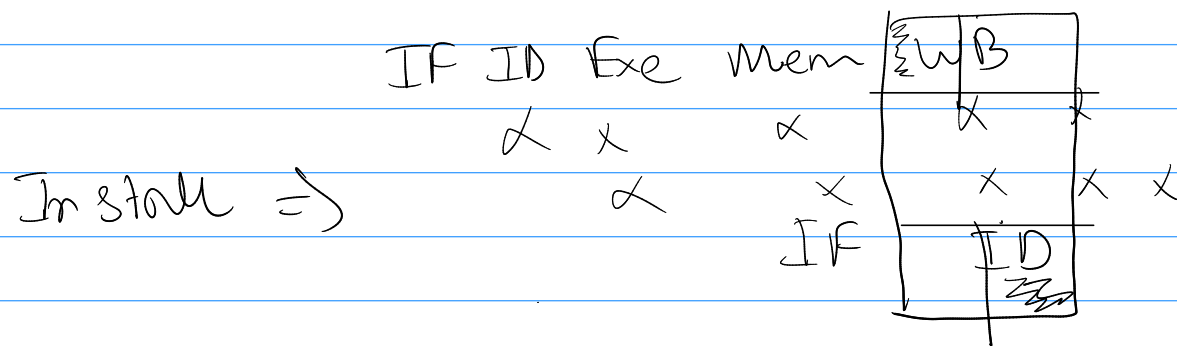
When the CPU fetches the next instruction in IF stage, it doesn't know if the current instruction is a branch (e.g., BEQ, JAL, etc.) that will change the PC (program counter).

So it may fetch the wrong instruction, causing a hazard.

Hazard Solution

1. Forwarding - Set a connect where the value is getting calculated
2. Stalling - Set idle for the data is being updated
3. Code scheduling - put the instruction with unique oprands in stalled cycle

Priority wise -> Forwarding, Stalling, Code scheduling



Question 04

Suppose you have a single instruction;

Add X21, X22, X23

Register read = 20s

Instruction Fetch = 20s

Memory Operation = 40s

Register write = 20s

ALU Operation = 40s

Single $\Rightarrow$ Ins Fetch $\Rightarrow 20$
Ins Decode $\Rightarrow 20$
Execution $\Rightarrow 40$
Mem $\Rightarrow \times$
Write $\Rightarrow 20$
 $\Rightarrow 100s$

Suppose you have a pipelined setup and a single-cycle setup,

Which one will run the instruction faster?

C1 C01	C2 C02	C3 C03	C4 C04	C5 C05
IF	ID	Exe	Mem	WB
$40 \times 5 \Rightarrow 200s$				
$\therefore$ Single-cycle will run faster				

Add X21, X22, X23 ✓

Sub X21, X21, X24 ✓

Mul X22, X21, X26

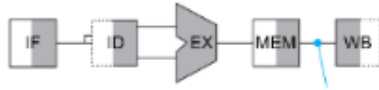
LD X22, 0(X21)



CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11	CC12
IF	ID	Execution	Mem	WB							
	x	x	x	x							
		x	x	x	x						
			IF	ID	Exc	Mem	WB				
				x	x	x	x	x			
					x	x	x	x	x		
						IF	ID	Exc	Mem	WB	
							IF	ID	Exc	Mem	WB

Question - 30 :

The instructions provided below are executed sequentially within a **pipelined** datapath that is divided into five stages, as depicted in the following diagram. In the diagram, the gray shaded areas represent the periods when tasks are actively being processed in each stage, whereas the unshaded (white) areas indicate inactive portions of each stage.



Instruction
Add X21, X22, X23
Sub X21, X20, X24
LD X22, 0(X22)

- Will any data hazards occur during the execution of this instruction set? If so, identify the number of data hazards and clearly circle them.
- How many clock cycles are necessary to run this instruction set?

LD X22, 0(X22)

Total clock cycle $\Rightarrow 7$

[illegible]

Question - 31 :

The following are the time requirements for various stages of a datapath:

1. Instruction Fetch: 25 ps
2. Instruction Decode: 10 ps
3. Register Read: 15 ps
4. Arithmetic or Logical task execution: 20 ps
5. Data Memory Read: 20 ps
6. Data Memory Write: 20 ps
7. Register Write: 25 ps

You are given the following instruction set, executed sequentially:

1. Add X21, X22, X23
2. Sub X21, X23, X24
3. Mul X22, X22, X26
4. LD X22, 0(X20)

Suppose this instruction set is executed sequentially in a single cycle datapath. Answer the following:

- a. Calculate the actual time required to execute each instruction.
- b. Determine the clock period of this system.
- c. Calculate the number of clock cycles needed to execute this instruction set.
- d. Determine the total time required to execute this instruction set.

Now, assume the instruction set is executed sequentially in an ideal pipelined datapath with 5 stages.

Answer the following:

- a. Calculate the actual time required to execute each instruction.
- b. Determine the clock period of this system.
- c. Calculate the number of clock cycles needed to execute this instruction set.
- d. Determine the total time required to execute this instruction set.

1. Add X21, X22, X23
2. Sub X21, X23, X24
3. Mul X22, X22, X26
4. LD X22, 0(X20)

1. Instruction Fetch: 25 ps
2. Instruction Decode: 10 ps
3. Register Read: 15 ps
4. Arithmetic or Logical task execution: 20 ps
5. Data Memory Read: 20 ps
6. Data Memory Write: 20 ps
7. Register Write: 25 ps

For single cycle datapath

a.

$$\text{Add} = 25 + 10 + 15 + 20 + 25 = 95$$

$$\text{Sub} = 25 + 10 + 15 + 20 + 25 = 95$$

$$\text{Mul} = 25 + 10 + 15 + 20 + 25 = 95$$

$$\text{LD} = 25 + 10 + 15 + 20 + 20 + 25 = 115$$

b. Clock period = 115

c. clock cycle = 4

d. Total time = $115 \times 4 = 460$ ps

For pipeline datapath

1. Add X21, X22, X23
2. Sub X21, X23, X24
3. Mul X22, X22, X26
4. LD X22, 0(X20)

1. Instruction Fetch: 25 ps
2. Instruction Decode: 10 ps
3. Register Read: 15 ps
4. Arithmetic or Logical task execution: 20 ps
5. Data Memory Read: 20 ps
6. Data Memory Write: 20 ps
7. Register Write: 25 ps

CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8			
IF	ID	Exe	Mem	WB						
	IF	ID	Exe	Mem	WB					
		IF	ID	Exe	Mem	WB				
			IF	ID	Exe	Mem	WB			

a) actual time $\Rightarrow 25 \times 5 \Rightarrow 125$ ps

b) clock period $\Rightarrow 25$ ps

c) clock cycle $\Rightarrow 8$

d) Total time $\Rightarrow 25 \times 8 \Rightarrow 200$ ps

Avg. CPI $\Rightarrow \frac{8}{4}$

$\Rightarrow 2$

1. What is actual time in pipeline?

2. will mem time be counted in single cycle r-type

b. Given a machine code

0x00CB1A93

3

i. Find the RISC-V assembly code of the given machine code.

ii. Draw the datapath for the instruction you found in the previous question. You must mention the necessary control signals and proper labeling for all the components.

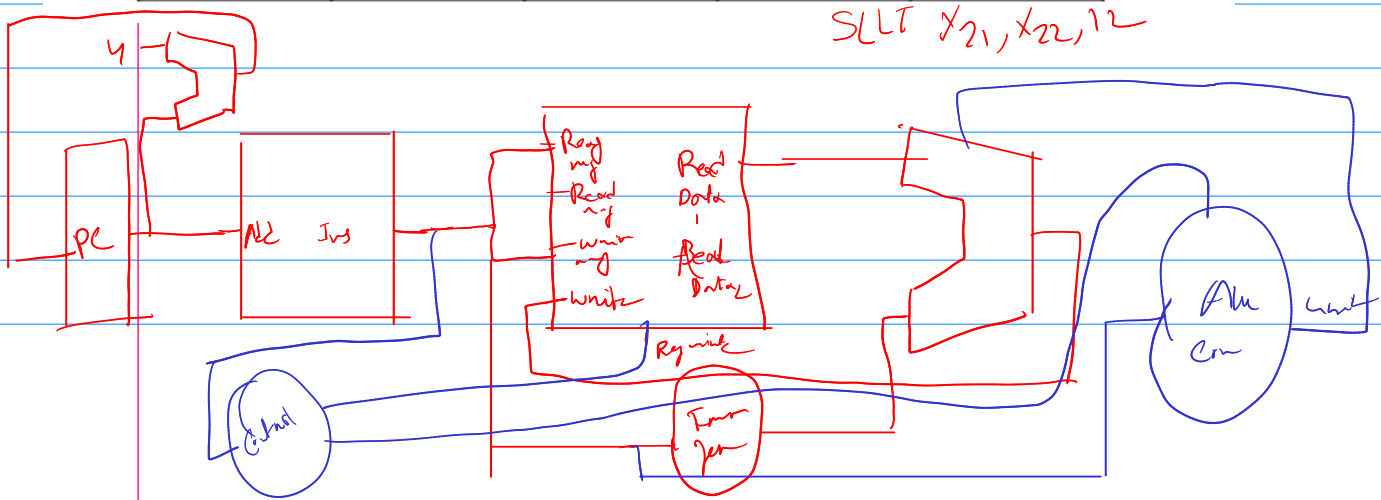
Ans.

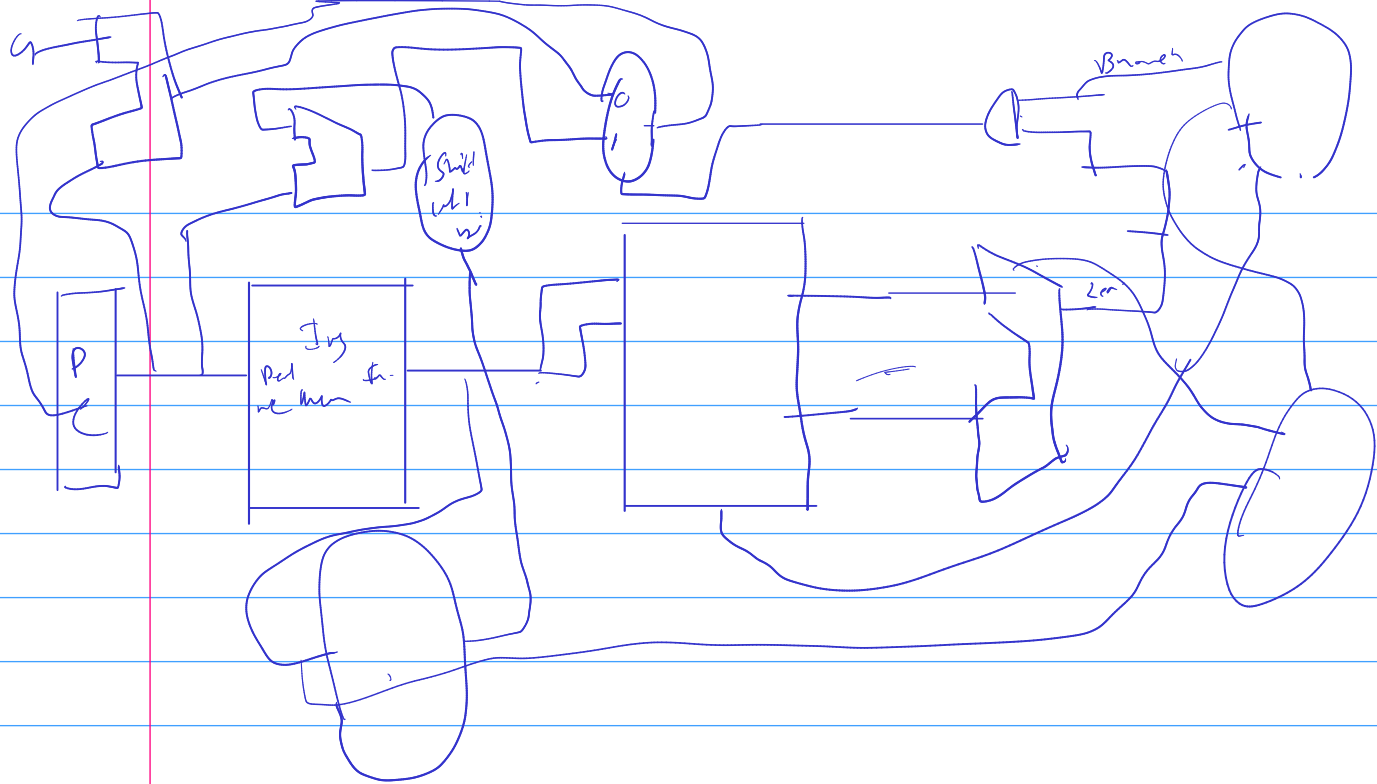
0000	0000	1100	1011	0001	1010	1001	0011
In			RS1	Inc3	rd		7
			10110	001	10101		0010011

$$S_{11}^{\text{in}} \alpha_{21}, \alpha_{22}, \alpha_{22}^*$$

Format	Instruction	Opcode	Funct3	Funct7/Funct6
R	ADD	0110011	001	0000000
	SUB	0110011	110	0000001
	AND	0110011	010	0000001
	OR	0110011	011	0000100
	SLL	0110011	101	0000100
I	LD	0000011	000	N/A
	LW	0000011	010	N/A
	LH	0000011	011	N/A
	LB	0000011	001	N/A
	ADDI	0010011	000	N/A
	SLLI	0010011	001	0000000
	SRLI	0010011	010	0000000
	ORI	0010011	011	N/A
S	SD	0100011	000	N/A
	SW	0100011	001	N/A
	SH	0100011	010	N/A
	SB	0100011	011	N/A

SLLT $x_{21}, x_{22}, 12$





4. CO4
- Explain why we need a memory hierarchy in our system. 2
 - Find the cache index, Tag, and offset bits for the memory address = 328, given that the Block size is 8 bits and the cache size is 32 bits. 1
 - "Choosing a large size for the cache line helps minimize the cache miss penalty." 2+1
Justify the validity of this statement with appropriate reasoning and examples.
 - Write a comparison between write-through and write-back techniques with proper examples. 1+1

$$\text{index bit} \Rightarrow \log_2(4) = 2$$

$$\text{Offset bit} \Rightarrow \log_2(1) = 0$$

$$\begin{array}{c} 101001009 \\ \hline \text{Tag} \quad \text{index} \end{array}$$

```
sub x7,x7,x7
sub x8,x8,x8
```

- c. Consider the C code below. **Generate** equivalent RISC-V code where i, n, and sum are in registers X6, X5, and X7. Also, A is a double word array, and the base register is X10: 5

```
sum = 0;
for (i = 0; i < n; i++) {
    if (A[i] / 2 == 0 && A[i] > 2 && i > 0)
        sum += A[i];
}
```

$i = x8$
 $n = x5$
 $sum = x7$

```
loop:
    bge x5,x7, exit
    slli x3, x8, 3
    add x9, x10, x3
    ld x21, 0(x9)
    srli x3, x21, 1
    beq x3, x0, con1
    beq x0,x0, inc
    con1:
        addi x20, x20, 2
        blt x20, x21, con2
        beq x0, x0, inc
    con2:
        bge x0, x8, inc
        add x7, x7, x21
        beq x0, x0, inc
    inc:
        addi x8, x8, 1
        beq x0,x0, loop
    exit:
```

1. a. **Convert** $-111011.11011 \times 2^{12}$ in 26-bit IEEE-754 format where, 2
- Size of the fraction field is 13 bits. 2
 - Size of the fraction field is 17 bits. 2
- Show** the equivalent Hex representation of your conversions. 2
- iii. Suppose you plan to use the converted number in a subsequent calculation where precision is crucial. Given a choice between the two IEEE formats (with 13-bit and 17-bit fraction fields), which format would provide a more accurate conversion for your calculation? Justify 2
- CO1 b. **Subtract** -6.55 from 15.21 using IEEE-754 single-precision floating-point representation. 3
- Consider** 5 decimal digits when you are converting from decimal to binary.
- c. For a 20-bit register with 1 bit for signs, 6 bits for exponents, and the rest for fractions, what will be the actual exponent range of the numbers that can be represented using IEEE 754 floating point representations? 2

$-1.1101111011 \times 2^{12}$

0-8191

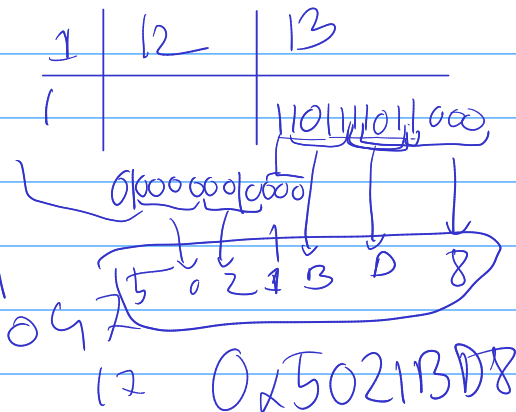
4095

1-4095, 8190-4095

-4095, 4095

Bias $\Rightarrow 2^{12-1} - 1$
 $\Rightarrow 2047$

$\Rightarrow 2064$



1

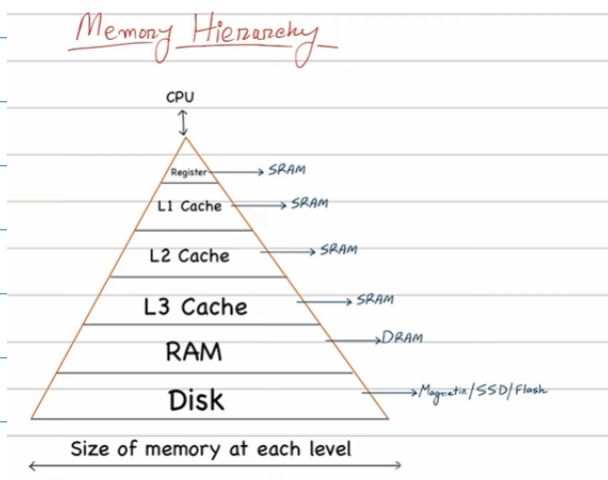
Chapter 5

Every memory have to characteristics

Temporal Locality: Items accessed recently are likely to be accessed again soon.

Spatial Locality: Items near those accessed recently are likely to be accessed soon.

Memory Hierarchy : Create a illusion of we have unlimited ammount of faster memory



Close to cpu

i. Size will decrease

ii. speed will increase

Difference between DRAM & SRAM

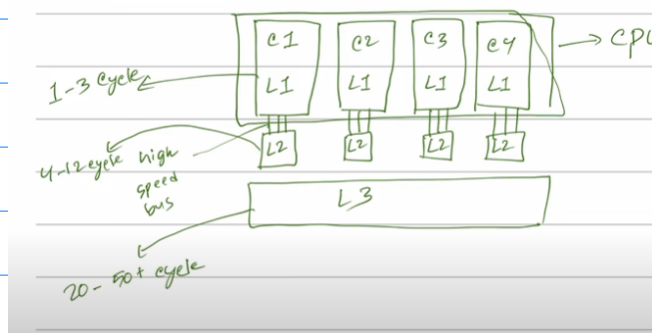
DRAM

- i. For storing 1 bit data need 1 transistor + 1 capacitor.
- ii. Power Consumption low.
- iii. Data retrival slower.
- iv. Cost efficient.

SRAM

- i. For storing 1 bit data need 6+ transistor.
- ii. Power Consumption higher.
- iii. Data retrival faster.
- iv. Cost higher.

L = Level



L1 cache is embeded with cpu core

L2 can be embeded with cpu or can be place outside of core with high speed bus

L3 cache placed outside of the core

L1 cache latency can be 1-3 cycle, L2 cache latency can be 4-12 cycle, L2 cache latency can be 20-50+ cycle

SRAM, DRAM

SRAM = Static Random Access Memory

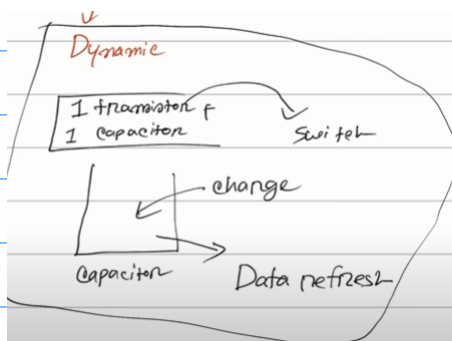
store data using transistor, don't need to refresh the memory until the electricity is passing

SRAM and DRAM both are volatile memory means both are capable of storing until the electricity is passing

DRAM = Dynamic Random Access Memory

store data using capacitor (charge), and transistor act as a switch,

After a certain period charge gets leaked, and we need to refresh that after certain time that's why it is called dynamic



Block/Line = The smallest unit of data that is transferred between main memory and cache called block or line, Block/line size depends on the system

Cache Hit/miss = Processor is requesting a data from cache if the data is present in the cache block is called hit.

If it is not present then it's called cache miss.

Hit ratio : Let's say cpu tried to access 5 data from cache and it got 3 data

Hit ratio = Number of hit/ number of access

also, Hit ratio = $1 - \text{miss ratio}$

Miss ratio = $1 - \text{hit ratio}$

Hit time = Time to check hit/miss + time to fetch that data

Hit is always small because, if data is hit that's mean it is somewhere in cache and we know that cache are small and faster

Miss penalty: Time to fetch the data from lower level(main memory) + Time to place the data in higher level(cache) + time to deliver the block to cpu

Ideal cache memory = increase the hit rate and decrease the miss penalty

1. Direct Mapped cache

2. Fully Associative

3. Set Associative

Direct Mapped cache

every cell of main memory is 8 bit

number of cell of block = $\text{block size} / 8\text{bit}$

block address bit = num of bit need to represent to number of block

Cache index = $\text{Block address} \bmod \text{number of block in the cache}$

Step 1 = Take the corresponding decimal value of memory address starting value.

Step 2 = mod with number of blocks

step 3 = transfer the mod value from decimal to binary

step 4= if the block is previously assigned it will overwrite with the new block

mod trick

13 mod 8

8 can be write in to 2^3

and binary value of 13 is 1101

now mod value will be 3 lsb bit of 1101

which is 101 = 5

Pros and cons of DMC

Pros:

1. Easy to implement in hardware
2. Lookups are faster

Cons:

1. Multiple main memory block might be mapped to the same cache index, overwriting the previous one

Address Subdivision

We divide address into 3 portion

Address= Tag bits, Index bits, offset bits

Index Bits= By seeing index bit we come to know that where I can find this address in cache. Index bit = $\log_2(\text{Number of cache blocks})$

Tag bit = Used to check if the block in the cach is correct or not.

Tag bit = Total add. size - index bit - offset bit

Offset bit = used to select exact byte within block

offset bit = $\log_2(\text{block size in byte})$

Valid bit = used to identify whether cache block contains data or not

Lookup process

Step 1= Identify index bit from address

step 2= then after go to that address check valid bit;

if valid bit = 1 ; valid data is present

if valid bit = 0 ; invalid data, cache miss

step 3 = match the tag bit

step 4 = if you want 1 byte data then using the offset bit and fetch the required data, if you want full block data then fetch the full block data.

EX. Memory address = 110101 ; Block size = 1 byte, number of block = 8; find out the index, tag and offset bit.

$$\text{Index} = \log_2(8) = 3 \text{ bit}$$

$$\text{offset} = \log_2(1) = 0 \text{ bit} \quad // \text{dont need any offset}$$

$$\begin{aligned} \text{Tag} &= \text{Total address size} - \text{idx} - \text{offset} \\ &= 6 - 3 - 0 = 3 \text{ bit} \end{aligned}$$

EX. Memory Address

1011 0101 0011 0000 0000 1010 1011 0010 1011 0101 0011 0000 0000 1010
1011 0010

Block size = 4 byte

Number of cache blocks = 1024

Find out the index, tag and offset bit.

$$\text{Index} = \log_2(1024) = 10 \text{ bit}$$

$$\text{offset} = \log_2(4) = 2 \text{ bit}$$

$$\text{Tag} = (16 * 4) - 10 - 2 = 52 \text{ bit}$$

1011 0101 0011 0000 0000 1010 1011 0010 1011 0101 0011 0000 0000 1010 1011 0010

Tag bit

Idx

offset

Tag bit = 1011 0101 0011 0000 0000 1010 1011 0010 1011 0101 0011 0000 0000

Index bit = 1010 1011 00

Offset bit = 10

Index bit calculation

Index = block address mod number of cache blocks

Ex. Given, block address = 25; number of cache blocks = 8
find the given cache index

$$25 = 11001, 8 = 2^3$$

$$\text{cache index} = 001$$

Ex. Given, memory address = 125; block size = 4 bytes; cache size = 8 blocks
find the index bits from the memory add.

$$\begin{aligned}\text{Block add} &= \text{mem add.} / \text{bytes per block} \\ &= 125 / 4 = 31.25 = \text{floor value} = 31\end{aligned}$$

$$\begin{aligned}\text{index bit} &= 31 \bmod 8 \\ &= 7 \text{ (111 binary)}\end{aligned}$$

Alternative

$$\text{index} = \log_2(8) = 3$$

$$\text{offset} = \log_2(4) = 2$$

$$\text{tag} = 7 - 3 - 2 = 2 \text{ bit}$$

1111101

tag = 11 (2 bits from msb)

index = 111 (3 bits after tag bit)

offset = 01 (2 bits from lsb)

Block size consideration

Benefits of larger block

1. Reduces miss Rate = Spatial locality $\rightarrow$ Access seq = $A, A+1, A+2$; the larger the block there is more data and fewer chances of miss

Issue:

1. Fewer blocks in cache: more data is fighting for fewer blocks if cache is fixed.
2. Cache pollution: if block size is large and you are using fewer bytes from then other unused bytes are polluting the cache
3. Increased miss penalty: if miss penalty happen, then larger the block larger data need to retrieval, to the time will increase.

Sol. TO reduce penalty time

Early restart: cpu requested a data, store the data sequentially when you get the required data then send the data

Critical word first: cpu requested a data, find the data and send the data then store the remaining block in background

Cache Miss

Instruction cache miss: The processor stop fetching instructions until the required instruction is brought from the memory . then restart the fetch process.

Data cache miss: It happens when a load instruction refers to a data that is not present in the cache. To solve this, CPU completes data access.

Cache hit on write operation

CPU wants to write(store) data to memory location and that location is already in the cache, the cpu can perform write quickly, this write hit is handled on the cache write policies.

1. Write through
2. Write back

1. Write through: on a write hit data need to be written on cache and memory simultaneously, this also keeps the memory up-to-date but this is little bit slower, because you have write in two place sol.

use buffer = write on cache and buffer later, transfer data from buffer to memory
problem : if buffer is full cpu will stall again.

2. Write back: Update data in cache block only and mark the block as dirty, when the dirty block is replaced write is back on the memory.
sol.

use buffer = instead of writing back on the memory, write it on a buffer. and later write it to memory in background
this help to avoid cpu stall

Cache miss on write operation.

A write miss occurs when the cpu wants to store(write) data to a memory location but that location is not present in the cache. Since the cache doesn't have the data block yet, it must decide what to do-- and this depends on the cache's write policy

1. write allocation

2. No- write allocation

1. write allocation: First take the block into the cache, then use write through or write back.

2. No- write allocation: Write in the main memory directly, and don't load the block into the cache

write-around = write-through + no write allocation

means:

It behaves like write-through when the block is in cache.

It behaves like no write allocation when the block is not in cache.

1. ✓ Write Hit (data is already in cache)

Here CPU can quickly update cache. But how should we handle main memory?

There are two policies:

i. Write-Through

Write happens in both cache and main memory.

Memory always up-to-date.

But slower, since every write must also go to main memory.

Solution: Use a write buffer → temporarily store writes so CPU doesn't stall.

☞ Example:

You want to update a diary and also your friend's diary at the same time.

Every time you write, you update both diaries immediately. Slow but always in sync.

ii. Write-Back

Write happens only in cache.

Mark that cache block as dirty (changed).

Actual update to main memory happens later, when that block is replaced.

Faster, fewer main memory writes.

Uses write buffer for efficiency.

☞ Example:

You only write in your diary. Later, when pages are full, you copy all changes into your friend's diary at once. Faster, but sometimes your diary and your friend's diary are not in sync until update happens.

2. ✗ Write Miss (data is NOT in cache)

Here CPU wants to write, but data block isn't in cache. What do we do?

i. Write Allocation (a.k.a. Fetch-on-write)

Bring the block from main memory into cache.

Then perform the write in cache (and possibly update memory depending on Write-Through / Write-Back).

Good if you expect future writes to the same block.

☞ Example:

You want to write in a notebook but don't have it. First, you bring the notebook to your desk (cache), then write in it.

ii. No-Write Allocation (a.k.a. Write-around)

Do not bring block into cache.

Just write directly to main memory.

Cache stays unchanged.

Good if you don't expect to use that block soon.

☞ Example:

You want to write something small. Instead of bringing the notebook to your desk, you just walk to the bookshelf (main memory) and write directly there.

You don't bother keeping it on your desk.

💠 Special Case: Write-Around

Combination: Write-Through + No-Write Allocation.

On write hit → update cache and memory.

On write miss → update only memory (skip cache).

Formula

- 
1. Hit ratio = Number of hit/ number of access
 2. Hit ratio = $1 - \text{miss ratio}$
 3. Miss ratio = $1 - \text{hit ratio}$
 4. Cache index = Block address mod number of block in the cache
 5. Tag bit = Total add. size - index bit - offset bit
 6. Index bit = $\log_2(\text{Number of cache blocks})$
 7. offset bit = $\log_2(\text{block size in byte})$ (Block size must have to be in byte)
 8. Block add = mem add./ bytes per block (Block size must have to be in byte)
Ans value will be ceiling value
 9. AMAT = Hit time + (Miss rate $\times$ Miss penalty)

Q-1: Given cache size = 4 blocks, Block Address = 129;
Find the cache index where this block will be mapped?

Q-2: Find the cache block address for the memory address = 321, if the block size is 10 bytes.

Q-3: Find the cache index for the memory address = 239, given that the Block size is 32 bits and the cache size is 64 bits.

Q-4: Find the cache index, Tag, and offset bits for the memory address = 297, given that the Block size is 8 bits and the cache size is 32 bits.

Q₁

Block address = 129 = 10000001

number of block = 4 = 2^2

Cache index = 01

Q₂

Memory address = 321

block size = 10 bytes

Cache block add = $321/10 = 32.1 = 32$

Q₃

mem add. = 239 = 11101111

block size = 32 bit

cache size = 64bit

number of blocks = $64/32 = 2$

index = $\log_2(2) = 1$

offset bit = $\log_2(4) = 2$

Tag bit = $8 - 1 - 2 = 5\text{bit}$

index bit = 1

Q₄

Memory add. = 297 = 100101001

Number of blocks = $32/8 = 4$

index = $\log_2(4) = 2\text{ bits} = 01$

offset = $\log_2(1) = 0\text{ bits}$

Tag bit = $9 - 2 - 0 = 7\text{ bits} = 1001010$

Q-5:

Cache table:

Index	Valid bit	Tag	Data(8 bits)
000	1	100	Data
001	0		
010	1	011 11	Data
011	1	001	Data
100	0 1	100	
101	1	000	Data
110	1	010 11	Data
111	0		

1. 100100
2. 111010
3. 010110
4. 100100
5. 111110

b. Hit ratio = $2/5 = 0.4$

miss ratio = $3/5 = 0.6$

c. AMAT = Hit time + (Miss rate $\times$ Miss penalty)

= $2 + (0.6 \times 5)$

= 5sec

d. clock cycle = $5/2 = 2.5$ cycles

a) For the following memory access, determine if it will be a hit or miss.

b) Calculate the hit and miss ratio.

c) Find the AMAT if the hit time is 2s and the miss penalty is 5s.

d) If the CPU has a 2s clock, then determine how many cycles are required per instruction.

a. index bit = $\log_2(8) = 3$

offset bit = $\log_2(1) = 0$

Tag bit = $6 - 3 - 0 = 3$ bit

1. For 100100 = Tag bit = 100

Index bit = 100

valid bit = 0 = miss

2. For 111010 = Tag bit = 111

Index bit = 010

valid bit = 1

Tag mismatched = miss

3. For 010110 = Tag bit = 010

Index bit = 110

valid bit = 1

Tag matched = Hit

4. For 100100 = Tag bit = 100

Index bit = 100

valid bit = 1

Tag matched = Hit

5. For 111110 = Tag bit = 111

Index bit = 110

valid bit = 1

Tag = mismatched = miss

Example: Given that the CPU has a 1ns clock, hit time = 1 cycle, miss penalty = 20 cycles, and l-cache miss rate = 5%, find the AMAT and how many cycles are required per instruction?

= Miss rate = $5/100 = 0.05$

Hit time = 1ns

Miss penalty = (20×1) ns = 20ns

AMAT = Hit time + (Miss rate $\times$ mis penalty)

= $1 + (0.05 \times 20)$

= 2 ns

2 ns = 2 clock cycle

Setup:

Cache Size = 8 blocks;

Block Size = 1 byte or 8 bits;

Offset bits size = $\log_2(1) = 0$; Hence, no Offset bits in the address.

Index	V	Tag	Data
000	N		
001	N		
010	N	7	16 [100 10]
011	N		
100	N		
101	N		
110	N	4	10 mem [10110]
111	N		

Access 1: Memory Address = 22 = 10110

Index = $\log_2(8) = 3 = 110$

offset = $\log_2(1) = 0$

Tag = $5 - 3 - 0 = 2 = 10$

Valid = 0 = Miss

Access 2: Memory Address = 26 = 11010

Index = 010

tag = 11

Valid = 0 = miss

Access 3: Memory Address = 22 = 10110

index = 110

tag = 10

hit

Access 4: Memory Address = 26 = 11010

index = 010

tag = 11

HIT

Access 5: Memory Address = 18 = 10010

index = 010

tag = 10

tag mismatched = miss